XCPC Templates

Skyclad Observer

Continue

October 19, 2026

Contents

1 代数结构 2
1.1 幺半群 2
1.2 幺半群作用 3
1.3 半群快速幂 4

2 模运算 4
2.1 Barrett 模乘 4
2.2 运算 4
2.3 模数类 5
2.4 all inverse 5

3 图论 6
3.1 Graph 类 6
3.2 线段树优化建图 7
3.3 最短路 7
3.3.1 Dijkstra 7
3.3.2 01-BFS 8
3.3.3 Bellman-Ford 9
3.3.4 SPFA 10
3.3.5 Floyd 10
3.3.6 路径重建 10
3.4 差分约束 11
3.5 拓扑排序 11
3.6 欧拉通路/回路 12
3.7 最小生成树 13
3.7.1 Kruskal 13
3.7.2 Prim 14
3.7.3 Borůvka 14
3.7.4 Kruskal 重构树 15
3.8 连通性 16
3.8.1 强连通分量 16
3.8.2 边双连通分量 16
3.8.3 点双连通分量与圆方树 16
3.9 2-SAT 17
3.10 树上问题 18
3.10.1 Tree 类 18
3.10.2 树链剖分 18
3.10.3 树上启发式合并 20
3.10.4 点分治 20
3.10.5 LCA 21
3.10.6 树上倍增 22
3.10.7 虚树 23
3.10.8 树上阿贝尔群 23
3.10.9 树上幺半群 24

4 网络流 25
4.1 最大流最小割 25
4.2 最小费用最大流 26

5 数论 28
5.1 扩展欧几里得算法 28
5.2 线性筛 28
5.3 莫比乌斯反演 29
5.4 欧拉函数 29
5.5 Miller-Rabin 30
5.6 Pollard-Rho 30
5.7 约数构造 31

6 组合数学 31
6.1 组合数 31
6.2 康托展开 32
6.3 逆序对 33
6.4 容斥原理 33
6.5 zeta 变换 34
6.6 Polyomino 35

7 数据结构 35
7.1 线段树 35
7.1.1 线段树 35
7.1.2 懒标记线段树 37
7.1.3 动态开点线段树 39
7.2 树状数组 41
7.2.1 树状数组 41
7.2.2 区间加、单点查树状数组 42
7.2.3 区间加、区间和树状数组 42
7.2.4 二维树状数组 43
7.3 倍增 43
7.4 并查集 44
7.5 稀疏表 45
7.6 01-Trie 45
7.7 笛卡尔树 46

8 线性代数 46
8.1 矩阵 46
8.2 高斯消元 48
8.3 GF(2) 空间 49
8.3.1 异或线性基 49
8.3.2 逆矩阵 50
8.3.3 转置 51
8.3.4 GF(2) 线性方程组 51

9 字符串 51
9.1 Manacher 51
9.2 KMP 52
9.3 Trie 52
9.4 Z 函数 52
9.5 AC 自动机 53

10 杂项 53
10.1 debug 53
10.2 template 54
10.3 二分答案 54
10.4 i128 55
10.5 日期 55

1 代数结构

1.1 幺半群

```cpp
// Template
// struct Monoid {
//     struct Type {
//         i64 v;
//     };
//     static Type e() {
//         return {};
//     }
//     static Type op(const Type& a, const Type& b) {
//         return {};
//     }
//     static Type from(const i64& x) {
//         return {x};
//     }
// };

template <typename Mono>
struct ReverseMono {
    using Type = typename Mono::Type;
    static Type e() {
        return Mono::e();
    }
    static Type op(const Type& a, const Type& b) {
        return Mono::op(b, a);
    }
};

template <typename T>
struct AddMono {
    using Type = T;
    static Type e() {
        return {0};
    }
    static Type op(const Type& a, const Type& b) {
        return {a + b};
    }
    static Type from(const i64& x) {
        return {x};
    }
    static Type inv(const Type& a) {
        return {-a};
    }
    static Type pow(const Type& a, i64 k) {
        return a * k;
    }
};

template <typename T>
struct MulMono {
    using Type = T;
    static Type e() {
        return {1};
    }
    static Type op(const Type& a, const Type& b) {
        return {a * b};
    }
    static Type from(const i64& x) {
        return {x};
    }
    static Type inv(const Type& a) {
        return T{1} / a;
    }
};

template <typename T>
struct MinMono {
    using Type = T;
    static Type e() {
        return {inf<T>};
    }
    static Type op(const Type& a, const Type& b) {
        return {min(a, b)};
    }
    static Type from(const i64& x) {
        return {x};
    }
    static Type pow(const Type& a, i64 k) {
        return (k == 0 ? e() : a);
    }
};

template <typename T>
struct MaxMono {
```

```
    using Type = T;
    static Type e() {
        return {-inf<T>};
    }
    static Type op(const Type& a, const Type& b) {
        return {max(a, b)};
    }
    static Type from(const i64& x) {
        return {x};
    }
    static Type pow(const Type& a, i64 k) {
        return (k == 0 ? e() : a);
    }
};

template <typename T>
struct GcdMono {
    using Type = T;
    static Type e() {
        return {0};
    }
    static Type op(const Type& a, const Type& b) {
        return {gcd(a, b)};
    }
    static Type from(const i64& x) {
        return {x};
    }
    static Type pow(const Type& a, i64 k) {
        return (k == 0 ? e() : a);
    }
};

template <typename T>
struct LcmMono {
    using Type = T;
    static Type e() {
        return {1};
    }
    static Type op(const Type& a, const Type& b) {
        return {lcm(a, b)};
    }
    static Type from(const i64& x) {
        return {x};
    }
    static Type pow(const Type& a, i64 k) {
        return (k == 0 ? e() : a);
    }
};

template <typename T>
struct XorMono {
    using Type = T;
    static Type e() {
        return {0};
    }
    static Type op(const Type& a, const Type& b) {
        return {a ^ b};
    }
    static Type inv(const Type& a) {
        return {a};
    }
    static Type power(const Type& a, i64 k) {
        return (k & 1) ? a : e();
    }
};
```

1.2 幺半群作用

```
// Template
// struct ActMono {
//     using Mono = Mono;
//     using Tag = Tag;
//     using X = Mono::Type;
//     using Y = Tag::Type;
//     static X act(const X& x, const Y& y, int len) {
//         return {};
//     }
// };

template <typename T>
struct SumAdd {
    using Mono = AddMono<T>;
    using Tag = AddMono<T>;
    using X =  Mono::Type;
    using Y = Tag::Type;
    static X act(const X& x, const Y& y, int len) {
        return {x + y * len};
```

```
    }
};

template <typename T>
struct SumMul {
    using Mono = AddMono<T>;
    using Tag = MulMono<T>;
    using X = Mono::Type;
    using Y = Tag::Type;
    static X act(const X& x, const Y& y, int len) {
        return {x * y};
    }
};

template <typename T>
struct MinAdd {
    using Mono = MinMono<T>;
    using Tag = AddMono<T>;
    using X = Mono::Type;
    using Y = Tag::Type;
    static X act(const X& x, const Y& y, int len) {
        return {x + y};
    }
};

template <typename T>
struct MaxAdd {
    using Mono = MaxMono<T>;
    using Tag = AddMono<T>;
    using X = Mono::Type;
    using Y = Tag::Type;
    static X act(const X& x, const Y& y, int len) {
        return {x + y};
    }
};
```

1.3 半群快速幂

```
template <typename Mono>
typename Mono::Type power(typename Mono::Type a, i64 b) {
    if constexpr (requires { Mono::pow(a, b); }) {
        return Mono::pow(a, b);
    } else {
        auto res = Mono::e();
        while (b > 0) {
            if (b & 1) {
                res = Mono::op(res, a);
            }
            a = Mono::op(a, a);
            b >>= 1;
        }
        return res;
    }
}
```

2 模运算

2.1 Barrett 模乘

```
class Barrett {
public:
    constexpr Barrett(uint32_t Mod) : mod_(Mod), mu_(static_cast<uint64_t>(-1) / Mod + 1) {}

    constexpr uint32_t mod() const {
        return mod_;
    }

    constexpr uint32_t multiply(uint32_t a, uint32_t b) const {
        uint64_t z = static_cast<uint64_t>(a) * b;
        uint64_t q = static_cast<uint64_t>((static_cast<__uint128_t>(z) * mu_) >> 64);
        uint64_t y = q * mod_;
        return static_cast<uint32_t>(z - y + (z < y ? mod_ : 0));
    }

private:
    uint32_t mod_;
    uint64_t mu_;
};
```

2.2 运算

```
i64 mul(i64 a, i64 b, i64 m) {
    return static_cast<i128>(a) * b % m;
}

i64 power(i64 a, i64 b, i64 m) {
    i64 res = 1 % m;
```

```
    while (b > 0) {
        if (b & 1) {
            res = mul(res, a, m);
        }
        a = mul(a, a, m);
        b >>= 1;
    }
    return res;
}

template <typename T, std::integral U>
constexpr T power(T a, U b) {
    assert(b >= 0);
    T res{1};
    while (b) {
        if (b & 1) {
            res *= a;
        }
        b >>= 1;
        a *= a;
    }
    return res;
}
```

2.3 模数类

```
template <u32 P>
struct Mint {
    u32 v;
    Mint(i64 v_ = 0) : v((v_ %= P) < 0 ? v_ + P : v_) {}
    Mint inv() const {
        i64 x, y;
        exgcd(v, P, x, y);
        return x;
    }
    Mint& operator+=(Mint rhs) & {
        v += rhs.v;
        if (v >= P) v -= P;
        return *this;
    }
    Mint& operator-=(Mint rhs) & {
        v -= rhs.v;
        if (v >= P) v += P;
        return *this;
    }
    Mint& operator*=(Mint rhs) & {
        v = u64(v) * rhs.v % P;
        return *this;
    }
    Mint& operator/=(Mint rhs) & {
        return *this *= rhs.inv();
    }
    friend Mint operator-(Mint rhs) { return rhs.v ? Mint(P - rhs.v) : rhs; }
    friend Mint operator+(Mint lhs, Mint rhs) { return lhs += rhs; }
    friend Mint operator-(Mint lhs, Mint rhs) { return lhs -= rhs; }
    friend Mint operator*(Mint lhs, Mint rhs) { return lhs *= rhs; }
    friend Mint operator/(Mint lhs, Mint rhs) { return lhs /= rhs; }
    friend bool operator==(Mint lhs, Mint rhs) { return lhs.v == rhs.v; }
    friend bool operator!=(Mint lhs, Mint rhs) { return lhs.v != rhs.v; }
    friend istream& operator>>(istream& is, Mint& v) { i64 x; is >> x; v = x;
return is; }
    friend ostream& operator<<(ostream& os, const Mint& v) { return os << v.v; }
};

constexpr u32 P = 998244353;
using Fp = Mint<P>;
```

2.4 all inverse

```
template <typename Fp>
vector<Fp> allInverse(const vector<Fp>& a) {
    for (auto& v : a) {
        assert(v != 0);
    }
    int n = a.size();
    vector<Fp> res(n + 1);
    res[0] = Fp{1};
    for (int i = 0; i < n; i++) {
        res[i + 1] = res[i] * a[i];
    }
    Fp inv = res[n].inverse();
    res.pop_back();
    for (int i = n - 1; i >= 0; i--) {
        res[i] *= inv;
        inv *= a[i];
    }
    return res;
}
```

3 图论

3.1 Graph 类

```cpp
template <typename T>
struct Edge {
    int u, v;
    T w;
};

template <typename T = int, bool directed = true>
struct Graph {
    int n;
    vector<Edge<T>> edges;
    vector<vector<pair<int, int>>> adj;

    Graph() : n(0) {}
    Graph(int n_) : n(n_), adj(n_) {}

    int add(int u, int v, T w = {1}) {
        int id = edges.size();
        edges.emplace_back(u, v, w);
        adj[u].emplace_back(v, id);
        if constexpr (!directed) {
            adj[v].emplace_back(u, id);
        }
        return id;
    }

    auto& operator[](int u) const {
        return adj[u];
    }
};

template <typename T>
Graph<T, true> reverse(const Graph<T, true>& g) {
    Graph<T, true> rg(g.n);
    for (auto& [u, v, w] : g.edges) {
        rg.add(v, u, w);
    }
    return rg;
}

template <typename T>
vector<int> degree(const Graph<T, false>& g) {
    vector<int> deg(g.n);
    for (auto& e : g.edges) {
        deg[e.u]++;
        deg[e.v]++;
    }
    return deg;
}

template <typename T>
pair<vector<int>, vector<int>> degree(const Graph<T, true>& g) {
    vector<int> in(g.n), out(g.n);
    for (auto& e : g.edges) {
        in[e.v]++;
        out[e.u]++;
    }
    return {in, out};
}

template <typename T, bool directed>
Graph<T, directed> induced(const Graph<T, directed>& g, const vector<int>& a) {
    int n = a.size();
    vector<int> id(g.n, -1);
    for (int i = 0; i < n; i++) {
        id[a[i]] = i;
    }
    Graph<T, directed> res(n);
    for (auto& e : g.edges) {
        if (id[e.u] != -1 && id[e.v] != -1) {
            res.add(id[e.u], id[e.v], e.w);
        }
    }
    return res;
}

template <typename T>
Graph<T, false> toUndigraph(const Graph<T, true>& g) {
    Graph<T, false> res(g.n);
    for (auto& e : g.edges) {
        res.add(e.u, e.v, e.w);
    }
    return res;
}
```

3.2 线段树优化建图

```cpp
template <typename T>
struct RangeGraph {
    int n, tot;
    vector<Edge<T>> edges;

    RangeGraph() {}
    RangeGraph(int n_) {
        init(n_);
    }

    void init(int n_) {
        n = n_;
        tot = 3 * n;
        edges.clear();
        for (int i = 2; i < 2 * n; i++) {
            add(out(i / 2), out(i), T{});
            add(in(i), in(i / 2), T{});
        }
    }

    int out(int i) {
        return (i >= n ? i - n : n + i);
    }
    int in(int i) {
        return (i >= n ? i - n : 2 * n + i);
    }

    int add(int u, int v, T w = {1}) {
        int id = edges.size();
        edges.emplace_back(u, v, w);
        return id;
    }

    void toRange(int u, int l, int r, T w = {1}) {
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) {
                add(u, out(l++), w);
            }
            if (r & 1) {
                add(u, out(--r), w);
            }
        }
    }

    void fromRange(int l, int r, int v, T w = {1}) {
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) {
                add(in(l++), v, w);
            }
            if (r & 1) {
                add(in(--r), v, w);
            }
        }
    }

    void RangeToRange(int l1, int r1, int l2, int r2, T w = {1}) {
        int u = tot++;
        fromRange(l1, r1, u, w);
        toRange(u, l2, r2, T{});
    }

    Graph<T, true> build() {
        Graph<T, true> g(tot);
        for (auto& [u, v, w] : edges) {
            g.add(u, v, w);
        }
        return g;
    }
};
```

3.3 最短路

3.3.1 Dijkstra

```cpp
template <typename T, bool directed>
pair<vector<T>, vector<int>> dijkstra(const Graph<T, directed>& g, int s) {
    int n = g.n;
    vector<T> dist(n, inf<T>);
    vector<int> pre(n, -1);
    using P =  pair<T, int>;
    priority_queue<P, vector<P>, greater<P>> pq;

    dist[s] = 0;
    pq.emplace(0, s);

    while (!pq.empty()) {
        auto [d, u] = pq.top();
```

```
        pq.pop();
        if (d != dist[u]) {
            continue;
        }
        for (auto [v, id] : g[u]) {
            T w = g.edges[id].w;
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pre[v] = u;
                pq.emplace(dist[v], v);
            }
        }
    }

    return {dist, pre};
}

template <typename T, bool directed>
pair<vector<T>, vector<int>> dijkstraDense(const Graph<T, directed>& g, int s) {
    int n = g.n;
    vector<T> dist(n, inf<T>);
    vector<int> pre(n, -1);
    vector<int> done(n, false);

    dist[s] = 0;

    for (int _ = 0; _ < n; _++) {
        int u = -1;
        for (int v = 0; v < n; v++) {
            if (!done[v] && (u == -1 || dist[v] < dist[u])) {
                u = v;
            }
        }
        if (u == -1 || dist[u] == inf<T>) {
            break;
        }
        done[u] = true;
        for (auto [v, id] : g[u]) {
            T w = g.edges[id].w;
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pre[v] = u;
            }
        }
    }

    return {dist, pre};
}

template <typename T, bool directed>
tuple<vector<T>, vector<int>, vector<int>> dijkstra(const Graph<T, directed>& g, vector<int>& s) {
    int n = g.n;
    vector<T> dist(n, inf<T>);
    vector<int> pre(n, -1), root(n, -1);
    using P =  pair<T, int>;
    priority_queue<P, vector<P>, greater<P>> pq;

    for (auto& u : s) {
        dist[u] = 0;
        root[u] = u;
        pq.emplace(0, u);
    }

    while (!pq.empty()) {
        auto [d, u] = pq.top();
        pq.pop();
        if (d != dist[u]) {
            continue;
        }
        for (auto [v, id] : g[u]) {
            T w = g.edges[id].w;
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pre[v] = u;
                root[v] = root[u];
                pq.emplace(dist[v], v);
            }
        }
    }

    return {dist, pre, root};
}
```

3.3.2 01-BFS

```
template <typename T, bool directed>
pair<vector<T>, vector<int>> bfs_01(const Graph<T, directed>& g, int s) {
```

```
    int n = g.n;
    vector<T> dist(n, inf<T>);
    vector<int> pre(n, -1);
    deque<int> que;

    dist[s] = 0;
    que.push_front(s);

    while (!que.empty()) {
        auto u = que.front();
        que.pop_front();
        for (auto [v, id] : g[u]) {
            T w = g.edges[id].w;
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pre[v] = u;
                if (w == 0) {
                    que.push_front(v);
                } else {
                    que.push_back(v);
                }
            }
        }
    }

    return {dist, pre};
}

template <typename T, bool directed>
tuple<vector<T>, vector<int>, vector<int>> bfs_01(const Graph<T, directed>& g,
vector<int>& s) {
    int n = g.n;
    vector<T> dist(n, inf<T>);
    vector<int> pre(n, -1), root(n, -1);
    deque<int> que;

    for (auto& u : s) {
        dist[u] = 0;
        root[u] = u;
        que.push_front(u);
    }

    while (!que.empty()) {
        auto u = que.front();
        que.pop_front();
        for (auto [v, id] : g[u]) {
            T w = g.edges[id].w;
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pre[v] = u;
                root[v] = root[u];
                if (w == 0) {
                    que.push_front(v);
                } else {
                    que.push_back(v);
                }
            }
        }
    }

    return {dist, pre, root};
}
```

3.3.3 Bellman-Ford

```
template <typename T, bool directed>
pair<vector<T>, vector<int>> bellmanFord(const Graph<T, directed>& g, int s) {
    int n = g.n;
    vector<T> dist(n, inf<T>);
    vector<int> pre(n, -1);

    dist[s] = 0;

    for (int round = 1;; round++) {
        bool upd = false;
        for (int u = 0; u < n; u++) {
            if (dist[u] == inf<T>) {
                continue;
            }
            for (auto [v, id] : g[u]) {
                T w = g.edges[id].w;
                if (dist[v] > dist[u] + w) {
                    dist[v] = dist[u] + w;
                    pre[v] = u;
                    upd = true;
                    if (round >= n) {
                        return {{}, {}};
```

```
                    }
                }
            }
        }
        if (!upd) {
            break;
        }
    }

    return {dist, pre};
}
```

3.3.4 SPFA

```
template <typename T, bool directed>
pair<vector<T>, vector<int>> spfa(const Graph<T, directed>& g, int s) {
    int n = g.n;
    vector<T> dist(n, inf<T>);
    vector<int> pre(n, -1);

    dist[s] = 0;

    queue<int> q;
    vector<char> inq(n, false);
    vector<int> cnt(n, 0);
    q.push(s);
    inq[s] = true;

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        inq[u] = false;
        for (auto [v, id] : g[u]) {
            T w = g.edges[id].w;
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pre[v] = u;
                if (!inq[v]) {
                    q.push(v);
                    inq[v] = true;
                    cnt[v]++;
                    if (cnt[v] >= n) {
                        return {{}, {}};
                    }
                }
            }
        }
    }

    return {dist, pre};
}
```

3.3.5 Floyd

```
template <typename T>
void floyd(vector<vector<T>>& dist) {
    int n = dist.size();
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (dist[i][k] < inf<T> && dist[k][j] < inf<T>) {
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
                }
            }
        }
    }
}

template <typename T, bool directed>
vector<vector<T>> floyd(const Graph<T, directed>& g) {
    int n = g.n;
    vector dist(n, vector<T>(n, inf<T>));
    for (int u = 0; u < n; u++) {
        dist[u][u] = 0;
        for (auto [v, id] : g[u]) {
            T w = g.edges[id].w;
            dist[u][v] = min(dist[u][v], w);
        }
    }
    floyd(dist);
    return dist;
}
```

3.3.6 路径重建

```
vector<int> restorePath(vector<int>& pre, int t) {
    vector<int> path = {t};
    while (pre[path.back()] != -1) {
```

```
        path.push_back(pre[path.back()]);
    }
    reverse(path.begin(), path.end());
    return path;
}
```

3.4 差分约束

```
template <typename T>
struct DifferenceConstraints {
    int n, s;
    Graph<T, true> graph;

    DifferenceConstraints() {}
    DifferenceConstraints(int n_) {
        init(n_);
    }

    void init(int n_) {
        n = n_;
        s = n;
        graph = Graph<T, true>(n + 1);
        for (int u = 0; u < n; u++) {
            graph.add(s, u);
        }
    }

    // x_v - x_u <= w
    void constraint(int u, int v, T w) {
        graph.add(u, v, w);
    }

    // x_v - x_u >= w
    void lowerBound(int u, int v, T w) {
        graph.add(v, u, -w);
    }

    // x_v - x_u = w
    void equality(int u, int v, T w) {
        constraint(u, v, w);
        lowerBound(u, v, w);
    }

    pair<bool, vector<T>> solve() {
        auto [dist, pre] = bellmanFord<T>(graph, s);
        if (dist.empty()) {
            return {false, {}};
        }
        dist.pop_back();
        return {true, dist};
    }
};
```

3.5 拓扑排序

```
template <typename T, bool directed>
vector<int> toposort(const Graph<T, directed>& g) {
    int n = g.n;
    vector<int> topo, deg(n, 0);
    for (int u = 0; u < n; u++) {
        for (auto [v, id] : g[u]) {
            deg[v]++;
        }
    }
    for (int u = 0; u < n; u++) {
        if (deg[u] == 0) {
            topo.push_back(u);
        }
    }
    for (int i = 0; i < int(topo.size()); i++) {
        int u = topo[i];
        for (auto [v, id] : g[u]) {
            if (--deg[v] == 0) {
                topo.push_back(v);
            }
        }
    }
    return (int(topo.size()) != n ? vector<int>() : topo);
}

template <typename T, bool directed>
vector<int> lexMinToposort(const Graph<T, directed>& g) {
    int n = g.n;
    vector<int> topo, deg(n, 0);
    for (int u = 0; u < n; u++) {
        for (auto [v, id] : g[u]) {
            deg[v]++;
        }
```

```
    }
    priority_queue<int, vector<int>, greater<int>> pq;
    for (int u = 0; u < n; u++) {
        if (deg[u] == 0) {
            pq.push(u);
        }
    }
    while (!pq.empty()) {
        int u = pq.top();
        pq.pop();
        topo.push_back(u);
        for (auto [v, id] : g[u]) {
            if (--deg[v] == 0) {
                pq.push(v);
            }
        }
    }
    return (int(topo.size()) != n ? vector<int>() : topo);
}
```

3.6 欧拉通路/回路

给定起点 s，还需要满足所有有边的顶点属于同一个连通分量，且 s 不能是孤立点。

无向图：

- 奇度点的数量一定是偶数，因此不会出现只有 1 个奇度点的情况。
- 有 0 个奇度点时，存在从任意有边顶点 s 出发的欧拉回路。
- 有 2 个奇度点时，s 必须是其中一个，存在从 s 出发的欧拉通路。
- 奇度点超过 2 个时不存在欧拉通路。

有向图：

- 若所有顶点满足 $\text{in} = \text{out}$，存在从任意有边顶点 s 出发的欧拉回路。
- 若恰有一个顶点满足 $\text{out} = \text{in} + 1$，它必须是 s；同时恰有一个顶点满足 $\text{in} = \text{out} + 1$，存在从 s 出发的欧拉通路。
- 其他情况不存在欧拉通路。

```
// 0: no Eulerian walk, 1: Eulerian path, 2: Eulerian circuit
template <typename T, bool directed>
int isEulerian(const Graph<T, directed>& g, int s = -1) {
    int n = g.n;
    if (g.edges.empty()) {
        return 2;  // 无边：任意点都有一条长度 0 的平凡欧拉回路
    }
    vector<int> deg(n);
    DSU dsu(n);
    for (auto& e : g.edges) {
        dsu.merge(e.u, e.v);
        deg[e.u]++;
        deg[e.v] += (directed ? -1 : 1);
    }
    int root = g.edges[0].u;
    for (auto& e : g.edges) {
        if (!dsu.same(root, e.u)) {
            return 0;
        }
    }
    if (s != -1 && !dsu.same(root, s)) {
        return 0;
    }
    int diff = 0;
    for (int u = 0; u < n; u++) {
        diff += directed ? abs(deg[u]) : (deg[u] & 1);
    }
    if (diff == 0) {
        return 2;
    }
    if (diff != 2) {
        return 0;
    }
    if (s == -1) {
        return 1;
    }
    return (directed ? deg[s] == 1 : (deg[s] & 1));
}

template <typename T, bool directed>
bool isEulerianPath(const Graph<T, directed>& g, int s = -1) {
    return isEulerian(g, s) >= 1;
}
template <typename T, bool directed>
bool isEulerianCircuit(const Graph<T, directed>& g, int s = -1) {
    return isEulerian(g, s) == 2;
}

template <typename T, bool directed>
pair<vector<int>, vector<int>> findEulerian(const Graph<T, directed>& g, int s =
-1) {
    int n = g.n;
    int m = g.edges.size();
```

```
    assert(n > 0);
    assert(s == -1 || (0 <= s && s < n));
    if (s == -1) {
        vector<int> deg(n);
        for (auto& e : g.edges) {
            deg[e.u]++;
            deg[e.v] += (directed ? -1 : 1);
        }
        if constexpr (directed) {
            s = max_element(deg.begin(), deg.end()) - deg.begin();
            if (deg[s] == 0) {
                s = m == 0 ? 0 : g.edges[0].u;
            }
        } else {
            s = (m == 0 ? 0 : g.edges[0].u);
            for (int u = 0; u < n; u++) {
                if (deg[u] & 1) {
                    s = u;
                    break;
                }
            }
        }
    }
    if (m == 0) {
        return {{s}, {}};
    }
    vector<int> bal(n), ptr(n), vs, es;
    vector<char> used(m, false);
    vector<pair<int, int>> stk = {{s, -1}};
    bal[s]++;
    while (!stk.empty()) {
        auto [u, eid] = stk.back();
        if (ptr[u] == int(g.adj[u].size())) {
            vs.push_back(u);
            if (eid != -1) {
                es.push_back(eid);
            }
            stk.pop_back();
        } else {
            auto [v, id] = g.adj[u][ptr[u]++];
            if (used[id]) {
                continue;
            }
            bal[u]--;
            bal[v]++;
            used[id] = true;
            stk.emplace_back(v, id);
        }
    }
    for (auto& x : bal) {
        if (x < 0) {
            return {{}, {}};
        }
    }
    if (int(vs.size()) != m + 1) {
        return {{}, {}};
    }
    reverse(vs.begin(), vs.end());
    reverse(es.begin(), es.end());
    return {vs, es};
}
```

3.7 最小生成树

3.7.1 Kruskal

```
template <typename T>
pair<vector<int>, T> kruskal(Graph<T, false>& g) {
    vector<int> ord(g.edges.size());
    iota(ord.begin(), ord.end(), 0);
    sort(ord.begin(), ord.end(), [&](int i, int j) {
        return g.edges[i].w < g.edges[j].w;
    });

    T cost = 0;
    vector<int> res;
    DSU dsu(g.n);

    for (auto& id : ord) {
        auto& e = g.edges[id];
        if (dsu.merge(e.u, e.v)) {
            res.push_back(id);
            cost += e.w;
        }
    }

    return {res, cost};
```

```
}

```

3.7.2 Prim

```
template <typename T, typename F>
vector<Edge<T>> prim(int n, F&& cost) {
    vector<Edge<T>> res;
    vector<T> wei(n, inf<T>);
    vector<int> pre(n);

    auto add = [&](int u) -> void {
        for (int v = 0; v < n; v++) {
            if (pre[v] == -1) {
                continue;
            }
            T w = cost(u, v);
            if (wei[v] > w) {
                wei[v] = w;
                pre[v] = u;
            }
        }
        wei[u] = inf<T>;
        pre[u] = -1;
    };

    add(0);
    for (int i = 0; i < n - 1; i++) {
        int u = min_element(wei.begin(), wei.end()) - wei.begin();
        res.emplace_back(pre[u], u, wei[u]);
        add(u);
    }

    return res;
}

template <typename T>
pair<vector<int>, T> prim(Graph<T, false>& g) {
    int n = g.n;
    vector<int> res, pre(n, -2);
    vector<T> wei(n, inf<T>);
    T cost = 0;
    auto add = [&](int u) -> void {
        for (auto& [v, id] : g[u]) {
            if (pre[v] == -1) {
                continue;
            }
            T w = g.edges[id].w;
            if (wei[v] > w) {
                wei[v] = w;
                pre[v] = id;
            }
        }
        wei[u] = inf<T>;
        pre[u] = -1;
    };
    add(0);
    for (int i = 0; i < n - 1; i++) {
        int u = min_element(wei.begin(), wei.end()) - wei.begin();
        if (wei[u] == inf<T>) {
            break;
        }
        res.push_back(pre[u]);
        cost += wei[u];
        add(u);
    }
    return {res, cost};
}
```

3.7.3 Borůvka

```
template <typename T>
pair<vector<int>, T> boruvka(Graph<T, false>& g) {
    auto& edges = g.edges;
    int n = g.n;
    T cost = 0;
    vector<int> res;
    DSU dsu(n);
    int cnt = n;

    while (cnt > 1) {
        vector<int> best(n, -1);
        int m = edges.size();

        for (int i = 0; i < m; i++) {
            auto& e = edges[i];
            int u = dsu.find(e.u);
            int v = dsu.find(e.v);
            if (u == v) {
```

```
                continue;
            }
            if (best[u] == -1 || e.w < edges[best[u]].w) {
                best[u] = i;
            }
            if (best[v] == -1 || e.w < edges[best[v]].w) {
                best[v] = i;
            }
        }

        bool upd = false;
        for (int u = 0; u < n; u++) {
            if (best[u] == -1) {
                continue;
            }
            auto& e = edges[best[u]];
            if (dsu.merge(e.u, e.v)) {
                res.push_back(best[u]);
                cost += e.w;
                cnt--;
                upd = true;
            }
        }
        if (!upd) {
            break;
        }
    }

    return {res, cost};
}
```

3.7.4 Kruskal 重构树

```
template <typename T>
struct KruskalTree {
    int n, root;
    vector<i64> val;
    vector<Edge<T>> edges;
    vector<vector<int>> adj;

    KruskalTree() : n(0) {}
    KruskalTree(Graph<T, false>& g) {
        init(g);
    }

    void init(Graph<T, false>& g) {
        n = g.n;
        root = n - 1;
        val.assign(2 * n - 1, 0);
        adj.assign(2 * n - 1, vector<int>{});
        edges = g.edges;
    }

    void build() {
        DSU dsu(2 * n - 1);
        vector<int> ord(int(edges.size()));
        iota(ord.begin(), ord.end(), 0);
        sort(ord.begin(), ord.end(), [&](int x, int y) {
            return edges[x].w < edges[y].w;
        });

        for (auto& id : ord) {
            auto& e = edges[id];
            int u = e.u, v = e.v;
            i64 w = e.w;

            if (!dsu.same(u, v)) {
                u = dsu.find(u);
                v = dsu.find(v);
                val[++root] = w;

                adj[root].push_back(u);
                adj[root].push_back(v);
                adj[u].push_back(root);
                adj[v].push_back(root);

                dsu.f[u] = root;
                dsu.f[v] = root;
                dsu.siz[root] = dsu.siz[u] + dsu.siz[v];

                if (root == 2 * n - 2) {
                    return;
                }
            }
        }
    }
};
```

3.8 连通性

3.8.1 强连通分量

```cpp
template <typename T>
pair<int, vector<int>> SCC(const Graph<T, true>& g) {
    int n = g.n;
    vector<int> stk, dfn(n, -1), low(n), comp(n, -1);
    int cur = 0, cnt = 0;
    auto dfs = [&](auto&& self, int u) -> void {
        dfn[u] = low[u] = cur++;
        stk.push_back(u);
        for (auto& [v, id] : g[u]) {
            if (dfn[v] == -1) {
                self(self, v);
                low[u] = min(low[u], low[v]);
            } else if (comp[v] == -1) {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (dfn[u] == low[u]) {
            int y;
            do {
                y = stk.back();
                comp[y] = cnt;
                stk.pop_back();
            } while (y != u);
            cnt++;
        }
    };
    for (int i = 0; i < n; i++) {
        if (dfn[i] == -1) {
            dfs(dfs, i);
        }
    }
    for (auto& u : comp) {
        u = cnt - 1 - u;
    }
    return {cnt, comp};
}
```

3.8.2 边双连通分量

```cpp
template <typename T>
pair<int, vector<int>> EBCC(const Graph<T, false>& g) {
    int n = g.n;
    vector<int> stk;
    vector<int> dfn(n, -1), low(n), comp(n, -1);
    int cur = 0, cnt = 0;
    auto dfs = [&](auto&& self, int u, int parentEdge) -> void {
        dfn[u] = low[u] = cur++;
        stk.push_back(u);
        for (auto [v, id] : g[u]) {
            if (id == parentEdge) {
                continue;
            }
            if (dfn[v] == -1) {
                self(self, v, id);
                low[u] = min(low[u], low[v]);
            } else {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (dfn[u] == low[u]) {
            int v;
            do {
                v = stk.back();
                stk.pop_back();
                comp[v] = cnt;
            } while (v != u);
            cnt++;
        }
    };
    for (int u = 0; u < n; u++) {
        if (dfn[u] == -1) {
            dfs(dfs, u, -1);
        }
    }
    return {cnt, comp};
}
```

3.8.3 点双连通分量与圆方树

```cpp
struct BlockCutTree {
    int n, tot;
    Graph<int, false> tree;

    BlockCutTree() {}
    template <typename T>
```

```
    BlockCutTree(const Graph<T, false>& g) {
        init(g);
    }

    template <typename T>
    void init(const Graph<T, false>& g) {
        n = g.n;
        tree = Graph<int, false>(n);
        vector<int> dfn(n, -1), low(n), stk;
        int cur = 0;
        // 点双的 >= 判定允许父边参与 low 更新，自环不影响结果。
        auto dfs = [&](auto&& self, int u) -> void {
            dfn[u] = low[u] = cur++;
            stk.push_back(u);
            for (auto [v, id] : g[u]) {
                if (dfn[v] == -1) {
                    self(self, v);
                    low[u] = min(low[u], low[v]);
                    if (low[v] >= dfn[u]) {
                        int b = tree.n++;
                        tree.adj.emplace_back();
                        tree.add(u, b);
                        int x;
                        do {
                            x = stk.back();
                            stk.pop_back();
                            tree.add(x, b);
                        } while (x != v);
                    }
                } else {
                    low[u] = min(low[u], dfn[v]);
                }
            }
        };
        for (int u = 0; u < n; u++) {
            if (dfn[u] == -1) {
                dfs(dfs, u);
                stk.pop_back();
            }
        }
        tot = tree.n;
    }

    bool isCut(int u) const {
        return tree[u].size() > 1;
    }
};
```

3.9 2-SAT

`addClause(u, f, v, g)` 加入一个析取子句：

$$(x_u = f) \vee (x_v = g).$$

这里 `x_i` 是第 `i` 个布尔变量，变量编号从 0 开始。`f` 和 `g` 决定文字的正负： `true` 表示正文字 x_i，`false` 表示负文字 $\neg x_i$。

例如： `addClause(3, true, 5, false)` 表示

$$x_3 \vee \neg x_5.$$

一个子句 $(A \vee B)$ 等价于两条蕴含边：

$$(\neg A) \to B, \quad (\neg B) \to A.$$

所以 `addClause(u, f, v, g)` 实际加入：

$$(x_u = \neg f) \to (x_v = g), \quad (x_v = \neg g) \to (x_u = f).$$

图中的编号约定是：$2i$ 表示变量 x_i 取假值，$2i+1$ 表示变量 x_i 取真值。 因此第一条边的起点是第一个文字的否定，终点是第二个文字本身；另一条边同理。

```
struct TwoSat {
    int n;
    Graph<int, true> graph;
    vector<int> ans;

    TwoSat(int n_) : n(n_), graph(2 * n), ans(n) {}

    // add clause: (x_u = f) ∨ (x_v = g)
    void addClause(int u, bool f, int v, bool g) {
        graph.add(2 * u + !f, 2 * v + g);
        graph.add(2 * v + !g, 2 * u + f);
    }

    // (x_u = f)
    void set(int u, bool f) {
        addClause(u, f, u, f);
    }

    // (x_u = f) -> (x_v = g)
    void implies(int u, int f, int v, int g) {
        addClause(u, !f, v, g);
    }

    pair<bool, vector<int>> satisfiable() {
```

```
        auto [cnt, id] = SCC(graph);
        for (int i = 0; i < n; i++) {
            if (id[2 * i] == id[2 * i + 1]) {
                return {false, {}};
            }
            ans[i] = id[2 * i] < id[2 * i + 1];
        }
        return {true, ans};
    }
};
```

3.10 树上问题

3.10.1 Tree 类

```
template <typename T>
struct Tree {
    Graph<T, false>& g;
    int n, root;
    vector<int> fa, pe, dep, siz, in, out, seq;
    vector<T> dist;

    Tree(Graph<T, false>& g, int rt = 0) : g(g) {
        init(rt);
    }

    void init(int rt = 0) {
        n = g.n;
        root = rt;
        assert(0 <= root && root < n);
        assert(int(g.edges.size()) == n - 1);
        fa.assign(n, -1);
        pe.assign(n, -1);
        dep.assign(n, 0);
        siz.assign(n, 1);
        in.resize(n);
        out.resize(n);
        seq.resize(n);
        dist.assign(n, T{});

        int cur = 0;
        auto dfs = [&](auto&& self, int u) -> void {
            in[u] = cur;
            seq[cur++] = u;
            for (auto [v, id] : g[u]) {
                if (v == fa[u]) {
                    continue;
                }
                fa[v] = u;
                pe[v] = id;
                dep[v] = dep[u] + 1;
                dist[v] = dist[u] + g.edges[id].w;
                self(self, v);
                siz[u] += siz[v];
            }
            out[u] = cur;
        };
        dfs(dfs, root);
    }

    int eToV(int i) {
        auto& e = g.edges[i];
        return (dep[e.u] > dep[e.v] ? e.u : e.v);
    }

    int vToE(int u) {
        return pe[u];
    }

    vector<int> subtree(int u) {
        return vector<int>(seq.begin() + in[u], seq.begin() + out[u]);
    }
};
```

3.10.2 树链剖分

```
template <typename T>
struct HLD {
    Tree<T>& tree;
    vector<int> heavy, top;

    HLD(Tree<T>& tree) : tree(tree) {
        build();
    }

    void build() {
        int n = tree.n;
        heavy.assign(n, -1);
```

```
        top.resize(n);
        for (int v = 0; v < n; v++) {
            int u = tree.fa[v];
            if (u == -1) {
                continue;
            }
            if (heavy[u] == -1 || tree.siz[v] > tree.siz[heavy[u]]) {
                heavy[u] = v;
            }
        }
        int cur = 0;
        auto dfs = [&](auto&& self, int u, int t) -> void {
            top[u] = t;
            tree.in[u] = cur;
            tree.seq[cur++] = u;
            if (heavy[u] != -1) {
                self(self, heavy[u], t);
            }
            for (auto [v, id] : tree.g[u]) {
                if (v == tree.fa[u] || v == heavy[u]) {
                    continue;
                }
                self(self, v, v);
            }
            tree.out[u] = cur;
        };
        dfs(dfs, tree.root, tree.root);
    }

    int lca(int u, int v) {
        while (top[u] != top[v]) {
            if (tree.dep[top[u]] < tree.dep[top[v]]) {
                swap(u, v);
            }
            u = tree.fa[top[u]];
        }
        return tree.dep[u] < tree.dep[v] ? u : v;
    }

    int dist(int u, int v) {
        return tree.dep[u] + tree.dep[v] - 2 * tree.dep[lca(u, v)];
    }

```

```
    int jump(int u, int k) {
        if (k > tree.dep[u]) {
            return -1;
        }
        int d = tree.dep[u] - k;
        while (tree.dep[top[u]] > d) {
            u = tree.fa[top[u]];
        }
        return tree.seq[tree.in[u] - tree.dep[u] + d];
    }

    int jump(int u, int v, int k) {
        int p = lca(u, v);
        int a = tree.dep[u] - tree.dep[p];
        int b = tree.dep[v] - tree.dep[p];
        if (k > a + b) {
            return -1;
        }
        if (k <= a) {
            return jump(u, k);
        }
        return jump(v, a + b - k);
    }

    bool isAncester(int u, int v) {
        return tree.in[u] <= tree.in[v] && tree.in[v] < tree.out[u];
    }

    int rootedParent(int root, int v) {
        if (root == v) {
            return v;
        }
        if (!isAncester(v, root)) {
            return tree.fa[v];
        }
        return jump(root, tree.dep[root] - tree.dep[v] - 1);
    }

    int rootedSize(int root, int v) {
        if (root == v) {
            return tree.n;
        }
        if (!isAncester(v, root)) {
```

```
            return tree.siz[v];
        }
        return tree.n - tree.siz[rootedParent(root, v)];
    }

    int rootedLca(int root, int u, int v) {
        return lca(root, u) ^ lca(root, v) ^ lca(u, v);
    }

    template <typename F>
    void forPath(int u, int v, bool edge, F&& f) {
        vector<pair<int, int>> down;
        while (top[u] != top[v]) {
            if (tree.dep[top[u]] > tree.dep[top[v]]) {
                f(tree.in[top[u]], tree.in[u] + 1, true);
                u = tree.fa[top[u]];
            } else {
                down.emplace_back(tree.in[top[v]], tree.in[v] + 1);
                v = tree.fa[top[v]];
            }
        }
        bool rev = tree.dep[u] > tree.dep[v];
        int l = min(tree.in[u], tree.in[v]) + edge;
        int r = max(tree.in[u], tree.in[v]) + 1;
        if (l < r) {
            f(l, r, rev);
        }
        for (int i = int(down.size()) - 1; i >= 0; i--) {
            auto [l, r] = down[i];
            f(l, r, false);
        }
    }
};

```

3.10.3 树上启发式合并

适用于静态树上的离线子树统计，如不同颜色数、颜色频次、众数等。保留重子树的数据，暴力补入轻子树的数据。

- `add(u)`：加入节点 `u` 的数据，不应依赖加入顺序。
- `query(u)`：当前数据恰好包含 `u` 的整棵子树，每个节点一次；据此记录答案，不破坏统计状态。
- `reset()`：清空全部统计状态，建议记录修改过的位置，避免每次清空整个值域。

调用前统计状态须为空，结束后也为空。`add` 共调用 $O(n\log n)$ 次，`query` 调用 n 次；若单次加入、查询为 $O(1)$，且清空成本可摊到加入操作上，总时间为 $O(n\log n)$。

```
template <typename T, typename F1, typename F2, typename F3>
void dsuOnTree(const HLD<T>& hld, F1&& add, F2&& query, F3&& reset) {
    const auto& tree = hld.tree;
    for (int i = tree.n - 1; i >= 0; i--) {
        int u = tree.seq[i];
        add(u);
        for (auto& [v, id] : tree.g[u]) {
            if (v == tree.fa[u] || v == hld.heavy[u]) {
                continue;
            }
            for (int j = tree.in[v]; j < tree.out[v]; j++) {
                add(tree.seq[j]);
            }
        }
        query(u);
        if (hld.top[u] == u) {
            reset();
        }
    }
}
```

3.10.4 点分治

回调 `f(fa, vt, pos)`：`vt[i]` 为原树编号，`vt[0]` 为重心；`fa[i]` 为局部父亲编号，`fa[0] = -1`，其余满足 `0 <= fa[i] < i`。从 $b=1$ 起，`[pos[b], pos[b+1])` 为各分支。只统计经过重心的路径，排除同分支点对；不应修改数组或保留引用。

当前块大小为 m，单次回调 $O(m)$ 时总时间 $O(n\log n)$，$O(m\log m)$ 时为 $O(n\log^2 n)$。初始化、清空也计入回调开销，避免每次扫描全局数组。

```
// f(fa, vt, pos)，局部编号 0 为重心，各分支占连续区间。
template <typename T, typename F>
void centroidDecompsition(const Tree<T>& tree, F&& f) {
    const auto& g = tree.g;
    vector<int> dead(tree.n), siz, fa, vt, pos;
    auto collect = [&](int u, int p) {
        int l = vt.size();
        vt.push_back(u);
        fa.push_back(p);
        for (int i = l; i < int(vt.size()); i++) {
            int u = vt[i];
            for (auto [v, id] : g[u]) {
                if (dead[v] || (fa[i] != -1 && v == vt[fa[i]])) {
                    continue;
                }
                vt.push_back(v);
```

```
                fa.push_back(i);
            }
        }
    };
    vector<int> roots{tree.root};
    for (int t = 0; t < int(roots.size()); t++) {
        vt.clear();
        fa.clear();
        collect(roots[t], -1);
        int m = vt.size(), c = -1;
        siz.assign(m, 1);
        for (int i = m - 1; i >= 0; i--) {
            if (siz[i] >= (m + 1) / 2) {
                c = vt[i];
                break;
            }
            siz[fa[i]] += siz[i];
        }
        dead[c] = true;
        vt = {c};
        fa = {-1};
        pos = {0, 1};
        for (auto [v, id] : g[c]) {
            if (!dead[v]) {
                roots.push_back(v);
                collect(v, 0);
                pos.push_back(vt.size());
            }
        }
        f(fa, vt, pos);
    }
}
```

3.10.5 LCA

```
template <typename T>
struct LCA {
    Tree<T>& tree;
    int logn;
    vector<vector<int>> fa;

    LCA(Tree<T>& tree) : tree(tree) {
        init();
    }

    void init() {
        int n = tree.n;
        logn = __lg(n);
        fa.assign(n, vector<int>(logn + 1));
        for (int u = 0; u < n; u++) {
            fa[u][0] = (tree.fa[u] == -1 ? u : tree.fa[u]);
        }
        for (int k = 1; k <= logn; k++) {
            for (int u = 0; u < n; u++) {
                fa[u][k] = fa[fa[u][k - 1]][k - 1];
            }
        }
    }

    int get(int u, int v) {
        if (tree.dep[u] < tree.dep[v]) {
            swap(u, v);
        }
        for (int i = logn; i >= 0; i--) {
            if (tree.dep[fa[u][i]] >= tree.dep[v]) {
                u = fa[u][i];
            }
        }
        if (u == v) {
            return u;
        }
        for (int i = logn; i >= 0; i--) {
            if (fa[u][i] != fa[v][i]) {
                u = fa[u][i];
                v = fa[v][i];
            }
        }
        return fa[u][0];
    }

    int dist(int u, int v) {
        int res = tree.dep[u] + tree.dep[v] - 2 * tree.dep[get(u, v)];
        return res;
    }
};

template <typename T>
```

```
struct FastLCA {
    struct DfnMin {
        struct Type {
            int dfn, fa;
        };
        static Type op(const Type& a, const Type& b) {
            return a.dfn < b.dfn ? a : b;
        }
    };

    Tree<T>& tree;
    SparseTable<DfnMin> st;

    FastLCA(Tree<T>& tree) : tree(tree) {
        init();
    }

    void init() {
        int n = tree.n;
        vector<typename DfnMin::Type> seq(n);
        for (int u = 0; u < n; u++) {
            int p = tree.fa[u] == -1 ? u : tree.fa[u];
            seq[tree.in[u]] = {tree.in[p], p};
        }
        st.init(seq);
    }

    int query(int u, int v) {
        if (u == v) {
            return u;
        }
        if (tree.in[u] > tree.in[v]) {
            swap(u, v);
        }
        return st.query(tree.in[u] + 1, tree.in[v] + 1).fa;
    }

    int dist(int u, int v) {
        return tree.dep[u] + tree.dep[v] - 2 * tree.dep[query(u, v)];
    }
};
```

3.10.6 树上倍增

```
template <typename Mono, typename T>
struct TreeLifting {
    using X = typename Mono::Type;

    Tree<T>& tree;
    BinaryLifting<Mono> lift;

    TreeLifting(Tree<T>& tree) : tree(tree) {
        init();
    }

    void init() {
        int n = tree.n;
        lift.init(n);
        for (int v = 0; v < n; v++) {
            if (v == tree.root) {
                lift.add(v, -1, Mono::e());
            } else {
                lift.add(v, tree.fa[v], Mono::from(tree.g.edges[tree.vToE(v)].w));
            }
        }
        lift.build();
    }

    int lca(int u, int v) {
        if (tree.dep[u] < tree.dep[v]) {
            swap(u, v);
        }
        u = lift.jump(u, tree.dep[u] - tree.dep[v]);
        if (u == v) {
            return u;
        }
        for (int k = lift.logn; k >= 0; k--) {
            if (lift.nxt[k][u] != lift.nxt[k][v]) {
                u = lift.nxt[k][u];
                v = lift.nxt[k][v];
            }
        }
        return lift.nxt[0][u];
    }

    pair<int, X> query(int u, int v) {
```

```
        int l = lca(u, v);
        X res = Mono::op(lift.query(u, tree.dep[u] - tree.dep[l]).second, lift.query(v, tree.dep[v] - tree.dep[l]).second);
        return {l, res};
    }
};
```

3.10.7 虚树

```
template <typename T>
struct VirtualTree {
    TreeLifting<AddMono<i64>, T>& tree;
    vector<int> vt;
    vector<vector<pair<int, T>>> adj;

    VirtualTree(TreeLifting<AddMono<i64>, T>& t) : tree(t) {}

    pair<vector<int>&, vector<vector<pair<int, T>>>&> build(vector<int>& h) {
        vt = h;
        auto cmp = [&](int u, int v) { return tree.tree.in[u] < tree.tree.in[v]; };
        sort(vt.begin(), vt.end(), cmp);
        int m = vt.size();
        for (int i = 1; i < m; i++) {
            vt.push_back(tree.lca(vt[i - 1], vt[i]));
        }
        sort(vt.begin(), vt.end(), cmp);
        vt.erase(unique(vt.begin(), vt.end()), vt.end());
        int n = vt.size();
        adj.resize(n);
        for (auto& e : adj) {
            e.clear();
        }
        for (int i = 0; i < n - 1; i++) {
            int p = tree.lca(vt[i], vt[i + 1]);
            int u = lower_bound(vt.begin(), vt.end(), p, cmp) - vt.begin();
            int v = i + 1;
            T w = tree.tree.dep[vt[v]] - tree.tree.dep[p];
            adj[u].emplace_back(v, w);
            adj[v].emplace_back(u, w);
        }

        return {vt, adj};
    }
};
```

3.10.8 树上阿贝尔群

```
template <typename T, typename Abel, bool edge = false>
struct TreeAbel {
    using X = typename Abel::Type;

    int n;
    HLD<T>& hld;
    Tree<T>& tree;
    Fenwick<Abel> sub;
    DualFenwick<Abel> path;

    TreeAbel(HLD<T>& hld_) : hld(hld_), tree(hld_.tree), n(hld_.tree.n), sub(n), path(n) {}
    TreeAbel(HLD<T>& hld_, const vector<X>& init) : hld(hld_), tree(hld_.tree), n(hld_.tree.n), sub(n), path(n) {
        assert(int(init.size()) == n - edge);
        for (int i = 0; i < int(init.size()); i++) {
            int u = i;
            if constexpr (edge) {
                u = tree.eToV(i);
            }
            path.add(tree.in[u], tree.out[u], init[i]);
            sub.add(tree.in[u], init[i]);
        }
    }

    X rootQuery(int u) {
        return path.query(tree.in[u]);
    }

    X pathQuery(int u, int v) {
        int l = hld.lca(u, v);
        X x = rootQuery(l);
        X res = Abel::op(rootQuery(u), rootQuery(v));
        res = Abel::op(res, Abel::inv(Abel::op(x, x)));
        if constexpr (!edge) {
            res = Abel::op(res, sub.rangeSum(tree.in[l], tree.in[l] + 1));
        }
        return res;
    }

    X subtreeQuery(int u, int root = -1) {
        if (root == -1) {
            root = tree.root;
```

```
        }
        if (root == u) {
            return sub.sum(n);
        }
        if (!hld.isAncester(u, root)) {
            return sub.rangeSum(tree.in[u] + edge, tree.out[u]);
        }
        int v = hld.jump(root, tree.dep[root] - tree.dep[u] - 1);
        return Abel::op(sub.rangeSum(0, tree.in[v]), sub.rangeSum(tree.out[v], n));
    }
};
```

3.10.9 树上幺半群

```
template <typename T, typename Mono, bool edge = false>
struct TreeMono {
    using X = typename Mono::Type;

    int n;
    HLD<T>& hld;
    Tree<T>& tree;
    SegmentTree<Mono> seg;
    SegmentTree<ReverseMono<Mono>> rseg;

    TreeMono(HLD<T>& hld) : n(hld.tree.n), hld(hld), tree(hld.tree) {
        build(vector<X>(n - edge, Mono::e()));
    }
    TreeMono(HLD<T>& hld, const vector<X>& init) : n(hld.tree.n), hld(hld), tree(hld.tree) {
        build(init);
    }

    void build(const vector<X>& init) {
        assert(int(init.size()) == n - edge);
        vector<X> a(n, Mono::e());
        for (int i = 0; i < int(init.size()); i++) {
            a[id(i)] = init[i];
        }
        seg.init(a);
        rseg.init(a);
    }

    void modify(int i, const X& x) {
        int p = id(i);
        seg.modify(p, x);
        rseg.modify(p, x);
    }

    void apply(int i, const X& x) {
        int p = id(i);
        modify(i, Mono::op(seg.rangeQuery(p, p + 1), x));
    }

    X pathQuery(int u, int v) {
        X ans = Mono::e();
        hld.forPath(u, v, edge, [&](int l, int r, bool rev) {
            X cur = (rev ? rseg.rangeQuery(l, r) : seg.rangeQuery(l, r));
            ans = Mono::op(ans, cur);
        });
        return ans;
    }

    X subtreeQuery(int u, int root = -1) {
        if (root == -1) {
            root = tree.root;
        }
        if (root == u) {
            return seg.rangeQuery(0, n);
        }
        if (!hld.isAncester(u, root)) {
            return seg.rangeQuery(tree.in[u] + edge, tree.out[u]);
        }
        assert(!edge);
        int v = hld.jump(root, tree.dep[root] - tree.dep[u] - 1);
        return Mono::op(seg.rangeQuery(0, tree.in[v]), seg.rangeQuery(tree.out[v], n));
    }

    X allQuery() {
        return seg.rangeQuery(0, n);
    }

    template <typename F>
    int maxPath(F&& pred, int u, int v) {
        X acc = Mono::e();
        if (edge && !pred(acc)) {
            return -1;
        }
```

```

        int ans = v;
        bool done = false;
        hld.forPath(u, v, edge, [&](int l, int r, bool rev) {
            if (done) {
                return;
            }
            X cur = rev ? rseg.rangeQuery(l, r) : seg.rangeQuery(l, r);
            X next = Mono::op(acc, cur);
            if (pred(next)) {
                acc = next;
                return;
            }
            int p = (rev ? rseg.minLeft(r, pred, acc) - 1 : seg.maxRight(l, pred,
acc));
            int w = tree.seq[p];
            if constexpr (edge) {
                ans = (rev ? w : tree.fa[w]);
            } else {
                ans = (w == u ? -1 : hld.jump(w, u, 1));
            }
            done = true;
        });
        return ans;
    }

    int id(int i) {
        int u = (edge ? tree.eToV(i) : i);
        return tree.in[u];
    }
};
```

4 网络流

4.1 最大流最小割

```
template <typename T>
struct MaxFlow {
    struct Edge_ {
        int to;
        T cap;
        Edge_(int to_, T cap_) : to(to_), cap(cap_) {}
    };

    int n;
    vector<Edge_> e;
    vector<vector<int>> adj;
    vector<int> cur, level;

    MaxFlow() {}
    MaxFlow(int n_) {
        init(n_);
    }

    void init(int n_) {
        n = n_;
        e.clear();
        adj.assign(n_, {});
        cur.resize(n_);
        level.resize(n_);
    }

    bool bfs(int s, int t) {
        level.assign(n, -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while (!q.empty()) {
            const int u = q.front();
            q.pop();
            for (auto& i : adj[u]) {
                auto [v, c] = e[i];
                if (c > 0 && level[v] == -1) {
                    level[v] = level[u] + 1;
                    if (v == t) {
                        return true;
                    }
                    q.push(v);
                }
            }
        }
        return false;
    }

    T dfs(int u, int t, T f) {
        if (u == t) {
            return f;
```

```
        }
        auto r = f;
        for (int& i = cur[u]; i < int(adj[u].size()); i++) {
            const int& j = adj[u][i];
            auto [v, c] = e[j];
            if (c > 0 && level[v] == level[u] + 1) {
                auto a = dfs(v, t, min(r, c));
                e[j].cap -= a;
                e[j ^ 1].cap += a;
                r -= a;
                if (r == 0) {
                    return f;
                }
            }
        }
        return f - r;
    }

    void addEdge(int u, int v, T c) {
        adj[u].push_back(e.size());
        e.emplace_back(v, c);
        adj[v].push_back(e.size());
        e.emplace_back(u, 0);
    }

    T flow(int s, int t) {
        T ans = 0;
        while (bfs(s, t)) {
            cur.assign(n, 0);
            ans += dfs(s, t, numeric_limits<T>::max());
        }
        return ans;
    }

    vector<bool> minCut() {
        vector<bool> c(n);
        for (int i = 0; i < n; i++) {
            c[i] = (level[i] != -1);
        }
        return c;
    }

    struct Edge {
        int from;
        int to;
        T cap;
        T flow;
    };

    vector<Edge> edges() {
        vector<Edge> a;
        for (int i = 0; i < e.size(); i += 2) {
            Edge x;
            x.from = e[i + 1].to;
            x.to = e[i].to;
            x.cap = e[i].cap + e[i + 1].cap;
            x.flow = e[i + 1].cap;
            a.push_back(x);
        }
        return a;
    }
};
```

4.2 最小费用最大流

最小费用流的每次增广都要在残量网络上找最短路，而残量网络里有反向边、费用是负的，Dijkstra 直接用不了。势能数组 h 就是用来把边权「平移」成非负的。

做法是把每条边的费用换成约化费用

$$c'(u,v) = c(u,v) + h(u) - h(v),$$

并且每轮求完最短路后令 $h(v) \leftarrow h(v) + \mathrm{dist}'(v)$，其中 dist' 是用约化费用算出的最短路。这样之后的约化费用依然非负—由三角不等式 $\mathrm{dist}'(v) \le \mathrm{dist}'(u) + c'(u,v)$ 整理一下，正好就是新的 $c'(u,v) \ge 0$。

代价是路径长度被平移了常量 $h(s) - h(t)$，但它不影响谁是最短路。代码里每轮累加的是 $h(t)$：因为 $h(s)$ 恒为 0，更新后的 $h(t)$ 恰好等于原网络中 $s \to t$ 的真实最短费用。

初始势能默认全 0，这就要求**初始边权全部非负**；如果原图本来就有负费用边，得根据图的特性自行构建 h 或者直接先跑一遍 SPFA 得到初始 h，再从参数传进来。

```
template <typename T>
struct MinCostFlow {
    struct Edge_ {
        int to;
        T cap;
        T cost;
        Edge_(int to_, T cap_, T cost_) : to(to_), cap(cap_), cost(cost_) {}
    };

    int n;
    vector<Edge_> e;
```

```
    vector<vector<int>> adj;
    vector<T> h, dist;
    vector<int> pre;

    bool dijkstra(int s, int t) {
        dist.assign(n, inf<T>);
        pre.assign(n, -1);
        priority_queue<pair<T, int>, vector<pair<T, int>>, greater<pair<T, int>>>
que;
        dist[s] = 0;
        que.emplace(0, s);
        while (!que.empty()) {
            auto [d, u] = que.top();
            que.pop();
            if (dist[u] != d) {
                continue;
            }
            for (auto& i : adj[u]) {
                int v = e[i].to;
                if (e[i].cap > 0 && dist[v] > d + h[u] - h[v] + e[i].cost) {
                    dist[v] = d + h[u] - h[v] + e[i].cost;
                    pre[v] = i;
                    que.emplace(dist[v], v);
                }
            }
        }
        return dist[t] != inf<T>;
    }

    MinCostFlow() {}
    MinCostFlow(int n_) {
        init(n_);
    }

    void init(int n_) {
        n = n_;
        e.clear();
        adj.assign(n, {});
        h.assign(n, {});
    }

    void addEdge(int u, int v, T cap, T cost) {
        adj[u].push_back(e.size());
        e.emplace_back(v, cap, cost);
        adj[v].push_back(e.size());
        e.emplace_back(u, 0, -cost);
    }

    void set(const vector<T>& potential) {
        h = potential;
    }

    pair<T, T> flow(int s, int t, T need = inf<T>) {
        T flow = 0;
        T cost = 0;
        while (flow < need && dijkstra(s, t)) {
            for (int i = 0; i < n; i++) {
                if (dist[i] != inf<T>) {
                    h[i] += dist[i];
                }
            }
            T aug = need - flow;
            for (int i = t; i != s; i = e[pre[i] ^ 1].to) {
                aug = min(aug, e[pre[i]].cap);
            }
            for (int i = t; i != s; i = e[pre[i] ^ 1].to) {
                e[pre[i]].cap -= aug;
                e[pre[i] ^ 1].cap += aug;
            }
            flow += aug;
            cost += aug * h[t];
        }
        return {flow, cost};
    }

    struct Edge {
        int from;
        int to;
        T cap;
        T cost;
        T flow;
    };

    vector<Edge> edges() {
        vector<Edge> a;
        for (int i = 0; i < e.size(); i += 2) {
            Edge x;
```

```
            x.from = e[i + 1].to;
            x.to = e[i].to;
            x.cap = e[i].cap + e[i + 1].cap;
            x.cost = e[i].cost;
            x.flow = e[i + 1].cap;
            a.push_back(x);
        }
        return a;
    }
};
```

5 数论

5.1 扩展欧几里得算法

```
i64 exgcd(i64 a, i64 b, i64& x, i64& y) {
    if (b == 0) {
        x = 1, y = 0;
        return a;
    }
    i64 d = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return d;
}

i64 inv(i64 a, i64 m) {
    i64 x, y;
    i64 d = exgcd(a, m, x, y);
    if (d != 1) {
        return -1;
    }
    return (x % m + m) % m;
}

bool solveLinear(i64 a, i64 b, i64 c, i64& x, i64& y) {
    i64 d = exgcd(a, b, x, y);
    if (c % d != 0) {
        return false;
    }
    i64 k = c / d;
    x *= k;
    y *= k;
    return true;
}
```

```

pair<i64, i64> solveMod(i64 a, i64 b, i64 m) {
    assert(m > 0);
    b *= -1;
    i64 x, y;
    i64 g = exgcd(a, m, x, y);
    if (g < 0) {
        g *= -1;
        x *= -1;
        y *= -1;
    }
    if (b % g != 0) {
        return {-1, -1};
    }
    x = x * (b / g) % (m / g);
    if (x < 0) {
        x += m / g;
    }
    return {x, m / g};
}

```

5.2 线性筛

```
vector<int> minp, primes;

void sieve(int n) {
    minp.assign(n + 1, 0);
    primes.clear();

    for (int i = 2; i <= n; i++) {
        if (minp[i] == 0) {
            minp[i] = i;
            primes.push_back(i);
        }

        for (auto p : primes) {
            if (i * p > n) {
                break;
            }
            minp[i * p] = p;
            if (p == minp[i]) {
                break;
            }
```

```
        }
    }
}
```

5.3 莫比乌斯反演

莫比乌斯反演用来在「按约数求和」和「按倍数求和」之间互转。

$\mu(n)$ 的定义：$n=1$ 时为 1；n 是 k 个互异素数之积时为 $(-1)^k$；含平方因子时为 0。一切从这条恒等式出发：

$$\sum_{d|n}\mu(d)=[n=1].$$

于是有两种方向的反演：

$$f(n)=\sum_{d|n}g(d)\Longrightarrow g(n)=\sum_{d|n}\mu(d)f\left(\frac{n}{d}\right),$$

$$f(n)=\sum_{n|d}g(d)\Longrightarrow g(n)=\sum_{n|d}\mu\left(\frac{d}{n}\right)f(d).$$

实际做题更常用的是「换条件」的用法：把 $[\gcd(i,j)=1]$ 换成 $\sum_{d|\gcd(i,j)}\mu(d)$，再交换求和顺序，条件就从 gcd 挪到了 d：

$$\sum_{i=1}^{n}\sum_{j=1}^{m}[\gcd(i,j)=1]=\sum_{d=1}^{\min(n,m)}\mu(d)\left\lfloor\frac{n}{d}\right\rfloor\left\lfloor\frac{m}{d}\right\rfloor.$$

右边只要 μ 的前缀和就能整除分块，μ 用下面的 `sieve()` 线性筛出来。

```
vector<int> minp, primes, mu;

void sieve(int n) {
    minp.assign(n + 1, 0);
    mu.assign(n + 1, 0);
    primes.clear();
    mu[1] = 1;

    for (int i = 2; i <= n; i++) {
        if (minp[i] == 0) {
            minp[i] = i;
            mu[i] = -1;
            primes.push_back(i);
        }

        for (auto p : primes) {
            if (i * p > n) {
                break;
            }
            minp[i * p] = p;
            if (p == minp[i]) {
                mu[i * p] = 0;
                break;
            } else {
                mu[i * p] = -mu[i];
            }
        }
    }
}
```

5.4 欧拉函数

$$\varphi(n)=n\prod_{i=1}^{k}\left(1-\frac{1}{p_i}\right).$$

两条常用性质：它是积性的，即 $\gcd(a,b)=1$ 时 $\varphi(ab)=\varphi(a)\varphi(b)$；并且 $\sum_{d|n}\varphi(d)=n$，也即 $\varphi(n)=\sum_{d|n}\mu(d)\frac{n}{d}$。

欧拉定理：$\gcd(a,m)=1$ 时 $a^{\varphi(m)}\equiv 1(\bmod m)$。竞赛里更常用的是扩展欧拉定理，它给指数降幂——$b\geq\varphi(m)$ 时对任意 a 都有

$$a^b\equiv a^{b\bmod\varphi(m)+\varphi(m)}(\bmod m).$$

线性筛求 φ 的递推：素数的 $\varphi(p)=p-1$；对 $i\cdot p$，若 $p\mid i$ 则 $\varphi(ip)=p\varphi(i)$，否则 $\varphi(ip)=(p-1)\varphi(i)$。见下面的 `sieve()`；只求单个数用 `Phi()`，复杂度 $O(\sqrt{n})$。

```
vector<int> minp, primes, phi;

// 线性筛（用于求解全部数的欧拉函数）
void sieve(int n) {
    minp.assign(n + 1, 0);
    phi.assign(n + 1, 0);
    primes.clear();
    phi[1] = 1;

    for (int i = 2; i <= n; i++) {
        if (minp[i] == 0) {
            minp[i] = i;
            primes.push_back(i);
            phi[i] = i - 1;
        }

        for (auto p : primes) {
            if (i * p > n) {
                break;
            }
            minp[i * p] = p;
            if (p == minp[i]) {
```

```
                phi[i * p] = phi[i] * p;
                break;
            } else {
                phi[i * p] = phi[i] * (p - 1);
            }
        }
    }
}

// 求解单个数的欧拉函数
int Phi(int n) {
    int res = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0) {
                n /= i;
            }
            res = res / i * (i - 1);
        }
    }
    if (n > 1) {
        res = res / n * (n - 1);
    }
    return res;
}

```

5.5 Miller-Rabin

```
bool isPrime(i64 n) {
    if (n < 2) {
        return false;
    }
    constexpr array<int, 10> smallPrimes = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29};
    for (auto& p : smallPrimes) {
        if (n % p == 0) {
            return n == p;
        }
    }
    if (n < 31 * 31) {
        return true;
    }
    constexpr array<int, 7> bases = {2, 325, 9375, 28178, 450775, 9780504,
1795265022};
    int s = __builtin_ctzll(n - 1);
    i64 d = (n - 1) >> s;
    for (auto& a : bases) {
        if (a % n == 0) {
            continue;
        }
        i64 cur = power(a, d, n);
        if (cur == 1) {
            continue;
        }
        bool witness = true;
        for (int r = 0; r < s; r++) {
            if (cur == n - 1) {
                witness = false;
                break;
            }
            cur = mul(cur, cur, n);
        }
        if (witness) {
            return false;
        }
    }
    return true;
}
```

5.6 Pollard-Rho

```
static mt19937_64
rng(static_cast<u32>(chrono::steady_clock::now().time_since_epoch().count()));

i64 rho(i64 n, i64 c) {
    auto f = [&](i64 x) {
        return static_cast<i64>((static_cast<i128>(x) * x + c) % n);
    };
    i64 x = 1, y = 2, z = 1, q = 1;
    i64 g = 1;
    constexpr i64 m = 127;
    for (i64 r = 1; g == 1; r <<= 1) {
        x = y;
        for (i64 i = 0; i < r; i++) {
            y = f(y);
        }
        for (i64 k = 0; k < r && g == 1; k += m) {
            z = y;
```

```
            for (i64 i = 0; i < min(m, r - k); i++) {
                y = f(y);
                q = mul(q, abs(x - y), n);
            }
            g = gcd(q, n);
        }
    }
    if (g == n) {
        do {
            z = f(z);
            g = gcd(abs(x - z), n);
        } while (g == 1);
    }
    return g;
}

i64 primeFactor(i64 n) {
    assert(n >= 2);
    if (n % 2 == 0) {
        return 2;
    }
    while (!isPrime(n)) {
        uniform_int_distribution<i64> pick(1, n - 1);
        n = rho(n, pick(rng));
    }
    return n;
}

vector<pair<i64, int>> factorize(i64 n) {
    vector<pair<i64, int>> pf;
    for (int p = 2; p < 100; p++) {
        if (p * p > n) {
            break;
        }
        if (n % p == 0) {
            int e = 0;
            do {
                n /= p;
                e++;
            } while (n % p == 0);
            pf.emplace_back(p, e);
        }
    }
    while (n > 1) {
        i64 p = primeFactor(n);
        int e = 0;
        do {
            n /= p;
            e++;
        } while (n % p == 0);
        pf.emplace_back(p, e);
    }
    sort(pf.begin(), pf.end());
    return pf;
}
```

5.7 约数构造

```
vector<i64> buildDivisors(const vector<pair<i64, int>>& factors) {
    vector<i64> divisors = {1};
    for (auto& [p, e] : factors) {
        int sz = divisors.size();
        for (int i = 0; i < sz; i++) {
            i64 x = divisors[i];
            for (int j = 0; j < e; j++) {
                x *= p;
                divisors.push_back(x);
            }
        }
    }
    return divisors;
}
```

6 组合数学

6.1 组合数

```
template <typename T>
struct Combination {
    int n;
    vector<T> fact_, invfact_, inv_;

    Combination() : n(0), fact_{1}, invfact_{1}, inv_{0} {}
    Combination(int n_) : Combination() {
        init(n_);
    }

    void init(int n_) {
```

```
        if (n_ <= n) {
            return;
        }
        fact_.resize(n_ + 1);
        invfact_.resize(n_ + 1);
        inv_.resize(n_ + 1);

        for (int i = n + 1; i <= n_; i++) {
            fact_[i] = fact_[i - 1] * i;
        }
        invfact_[n_] = fact_[n_].inv();
        for (int i = n_; i > n; i--) {
            invfact_[i - 1] = invfact_[i] * i;
            inv_[i] = invfact_[i] * fact_[i - 1];
        }
        n = n_;
    }

    T fact(int k) {
        if (k > n) {
            init(2 * k);
        }
        return fact_[k];
    }

    T invfact(int k) {
        if (k > n) {
            init(2 * k);
        }
        return invfact_[k];
    }

    T inv(int k) {
        if (k > n) {
            init(2 * k);
        }
        return inv_[k];
    }

    T binom(int n, int m) {
        if (n < m || m < 0) {
            return 0;
        }
        return fact(n) * invfact(m) * invfact(n - m);
    }

    T perm(int n, int m) {
        if (n < m || m < 0) {
            return 0;
        }
        return fact(n) * invfact(n - m);
    }

    T multicomb(int n, int m) {
        if (n < 0 || m < 0) {
            return 0;
        }
        if (n == 0) {
            return (m == 0);
        }
        return binom(n + m - 1, m);
    }

    T Catalan(int n) {
        if (n < 0) {
            return 0;
        }
        return binom(2 * n, n) * inv(n + 1);
    }
};
```

6.2 康托展开

```
vector<int> toLehmer(vector<int>& p) {
    int n = p.size();
    Fenwick<AddMono<int>> fen(n);
    vector<int> c(n);
    for (int i = n - 1; i >= 0; i--) {
        c[i] = fen.sum(p[i]);
        fen.add(p[i], 1);
    }
    return c;
}

vector<int> fromLehmer(vector<int>& c) {
    int n = c.size();
    vector<int> p(n);
```

```
    Fenwick<AddMono<int>> fen(n);
    for (int i = 0; i < n; i++) {
        fen.add(i, 1);
    }
    for (int i = 0; i < n; i++) {
        p[i] = fen.kth(c[i]);
        fen.add(p[i], -1);
    }
    return p;
}

i64 toRank(vector<int>& p) {
    int n = p.size();
    auto c = toLehmer(p);
    i64 fac = 1, rk = 0;
    for (int i = n - 1; i >= 0; i--) {
        rk += c[i] * fac;
        fac *= (n - i);
    }
    return rk;
}

vector<int> fromRank(int n, i64 rk) {
    vector<int> c(n);
    for (int i = 1; i <= n; i++) {
        c[n - i] = rk % i;
        rk /= i;
    }
    return fromLehmer(c);
}
```

6.3 逆序对

```
i64 invCount(vector<i64>& a, int l, int r, vector<i64>& tmp) {
    if (r - l <= 1) return 0;

    i64 res = 0;
    int mid = (l + r) / 2;
    res += invCount(a, l, mid, tmp);
    res += invCount(a, mid, r, tmp);

    int i = l, j = mid, k = l;
    while (i < mid && j < r) {
        if (a[i] <= a[j]) {
            tmp[k++] = a[i++];
        } else {
            tmp[k++] = a[j++];
            res += mid - i;
        }
    }
    while (i < mid) {
        tmp[k++] = a[i++];
    }
    while (j < r) {
        tmp[k++] = a[j++];
    }

    for (i = l; i < r; i++) {
        a[i] = tmp[i];
    }

    return res;
}
```

6.4 容斥原理

```
// cap(S) 表示 S 集合的交集大小

// 交集形式
template <typename T, typename F>
T ie(int n, F&& cap) {
    T ans{};
    for (int S = 1; S < (1 << n); S++) {
        T cur = cap(S);
        if (__builtin_popcount(S) & 1) {
            ans += cur;
        } else {
            ans -= cur;
        }
    }
    return ans;
}

// 补集形式
template <typename T, typename F>
T ieComp(int n, F&& cap) {
    T ans{};
```

```
    for (int S = 0; S < (1 << n); S++) {
        T cur = cap(S);
        if (__builtin_popcount(S) & 1) {
            ans -= cur;
        } else {
            ans += cur;
        }
    }
    return ans;
}

// 恰好满足 k 个性质的元素数量
template <typename T, typename F>
vector<T> ieExact(int n, F&& cap) {
    vector<T> N(n + 1, T{});
    for (int S = 0; S < (1 << n); S++) {
        int k = __builtin_popcount(S);
        N[k] += cap(S);
    }

    vector<T> binom(n + 2, T{}), E(n + 1, T{});
    binom[0] = T{1};
    for (int j = 0; j <= n; j++) {
        for (int k = 0; k <= j; k++) {
            T term = N[j] * binom[k];
            if ((j - k) & 1) {
                E[k] -= term;
            } else {
                E[k] += term;
            }
        }
        for (int k = j + 1; k >= 1; k--) {
            binom[k] += binom[k - 1];
        }
    }
    return E;
}

// 至少满足 k 个
template <typename T, typename F>
vector<T> ieLeast(int n, F&& cap) {
    auto E = ieExact<T>(n, cap);
    for (int k = n - 1; k >= 0; k--) {
        E[k] += E[k + 1];
    }
    return E;
}

// 至多满足 k 个
template <typename T, typename F>
vector<T> ieMost(int n, F&& cap) {
    auto E = ieExact<T>(n, cap);
    for (int k = 0; k < n; k++) {
        E[k + 1] += E[k];
    }
    return E;
}
```

6.5 zeta 变换

```
// 子集和：f[mask] = sum_{sub ⊆ mask} f[sub]
// 方向：小的 mask → 大的 mask（"吸收"不带的位）
template <typename T>
void subsetZeta(vector<T>& f) {
    int n = f.size();
    for (int i = 1; i < n; i <<= 1) {
        for (int mask = 0; mask < n; mask++) {
            if (mask & i) {
                f[mask] += f[mask ^ i];
            }
        }
    }
}

// 子集和的逆
template <typename T>
void subsetMobius(vector<T>& f) {
    int n = f.size();
    for (int i = 1; i < n; i <<= 1) {
        for (int mask = 0; mask < n; mask++) {
            if (mask & i) {
                f[mask] -= f[mask ^ i];
            }
        }
    }
}

```

```cpp
// 超集和：f[mask] = sum_{sup ⊇ mask} f[sup]
// 方向：大的 mask → 小的 mask（"分发"给不带这一位的 mask）
template <typename T>
void supersetZeta(vector<T>& f) {
    int n = f.size();
    for (int i = 1; i < n; i <<= 1) {
        for (int mask = 0; mask < n; mask++) {
            if (mask & i) {
                f[mask ^ i] += f[mask];
            }
        }
    }
}

// 超集和的逆
template <typename T>
void supersetMobius(vector<T>& f) {
    int n = f.size();
    for (int i = 1; i < n; i <<= 1) {
        for (int mask = 0; mask < n; mask++) {
            if (mask & i) {
                f[mask ^ i] -= f[mask];
            }
        }
    }
}
```

6.6 Polyomino

```cpp
struct Polyomino {
    using Cell = pair<int, int>;
    set<Cell> cells;
    Polyomino() {}
    Polyomino(const set<Cell>& cells_) : cells(cells_) {}
};

Polyomino normalize(const Polyomino& p) {
    if (p.cells.empty()) {
        return p;
    }
    int minX = p.cells.begin()->first;
    int minY = p.cells.begin()->second;
    for (auto& [x, y] : p.cells) {
        minX = min(minX, x);
        minY = min(minY, y);
    }
    Polyomino res;
    for (auto& [x, y] : p.cells) {
        res.cells.emplace(x - minX, y - minY);
    }
    return res;
}

Polyomino rotate(const Polyomino& p) {
    Polyomino res;
    for (auto& [x, y] : p.cells) {
        res.cells.emplace(y, -x);
    }
    return normalize(res);
}

Polyomino flip(const Polyomino& p) {
    Polyomino res;
    for (auto& [x, y] : p.cells) {
        res.cells.emplace(x, -y);
    }
    return normalize(res);
}
```

7 数据结构

7.1 线段树

7.1.1 线段树

```cpp
template <typename Mono>
struct SegmentTree {
    using X = typename Mono::Type;

    int n;
    vector<X> info;

    SegmentTree() : n(0) {}
    SegmentTree(const vector<X>& init_) {
        init(init_);
    }

    void init(const vector<X>& init_) {
        n = init_.size();
```

```
        int logn = __lg(n);
        info.assign(4 << logn, Mono::e());

        auto build = [&](auto&& self, int p, int l, int r) -> void {
            if (r - l == 1) {
                info[p] = init_[l];
                return;
            }
            int mid = (l + r) / 2;
            self(self, 2 * p, l, mid);
            self(self, 2 * p + 1, mid, r);
            pull(p);
        };

        build(build, 1, 0, n);
    }

    void pull(int p) {
        info[p] = Mono::op(info[2 * p], info[2 * p + 1]);
    }

    void modify(int p, int l, int r, int x, const X& v) {
        if (r - l == 1) {
            info[p] = v;
            return;
        }
        int mid = (l + r) / 2;
        if (x < mid) {
            modify(2 * p, l, mid, x, v);
        } else {
            modify(2 * p + 1, mid, r, x, v);
        }
        pull(p);
    }
    void modify(int p, const X& v) {
        modify(1, 0, n, p, v);
    }

    X rangeQuery(int p, int l, int r, int x, int y) {
        if (y <= l || r <= x) {
            return Mono::e();
        }
        if (x <= l && r <= y) {
            return info[p];
        }
        int mid = (l + r) / 2;
        return Mono::op(rangeQuery(2 * p, l, mid, x, y), rangeQuery(2 * p + 1, mid, r, x, y));
    }
    X rangeQuery(int l, int r) {
        return rangeQuery(1, 0, n, l, r);
    }

    template <typename F>
    int findFirst(int p, int l, int r, int x, int y, F&& pred) {
        if (y <= l || r <= x) {
            return -1;
        }
        if (x <= l && r <= y && !pred(info[p])) {
            return -1;
        }
        if (r - l == 1) {
            return l;
        }
        int mid = (l + r) / 2;
        int res = findFirst(2 * p, l, mid, x, y, pred);
        if (res == -1) {
            res = findFirst(2 * p + 1, mid, r, x, y, pred);
        }
        return res;
    }
    template <typename F>
    int findFirst(int l, int r, F&& pred) {
        return findFirst(1, 0, n, l, r, pred);
    }

    template <typename F>
    int findLast(int p, int l, int r, int x, int y, F&& pred) {
        if (y <= l || r <= x) {
            return -1;
        }
        if (x <= l && r <= y && !pred(info[p])) {
            return -1;
        }
        if (r - l == 1) {
            return l;
```

```
        }
        int mid = (l + r) / 2;
        int res = findLast(2 * p + 1, mid, r, x, y, pred);
        if (res == -1) {
            res = findLast(2 * p, l, mid, x, y, pred);
        }
        return res;
    }
    template <typename F>
    int findLast(int l, int r, F&& pred) {
        return findLast(1, 0, n, l, r, pred);
    }

    template <typename F>
    int maxRight(int l, F&& pred, X& sum) {
        int p = findFirst(l, n, [&](const X& x) {
            X v = Mono::op(sum, x);
            if (!pred(v)) {
                return true;
            }
            sum = v;
            return false;
        });
        return p == -1 ? n : p;
    }

    template <typename F>
    int minLeft(int r, F&& pred, X& sum) {
        int p = findLast(0, r, [&](const X& x) {
            X v = Mono::op(x, sum);
            if (!pred(v)) {
                return true;
            }
            sum = v;
            return false;
        });
        return p + 1;
    }
};
```

7.1.2 懒标记线段树

```
template <typename ActMono>
struct LazySegmentTree {
    using Mono = typename ActMono::Mono;
    using Tag = typename ActMono::Tag;
    using X = typename ActMono::X;
    using Y = typename ActMono::Y;

    int n;
    vector<X> info;
    vector<Y> tag;

    LazySegmentTree() : n(0) {}
    LazySegmentTree(const vector<X>& init_) {
        init(init_);
    }

    void init(const vector<X>& init_) {
        n = init_.size();
        int logn = __lg(n);
        info.assign(4 << logn, Mono::e());
        tag.assign(4 << logn, Tag::e());

        auto build = [&](auto&& self, int p, int l, int r) -> void {
            if (r - l == 1) {
                info[p] = init_[l];
                return;
            }
            int mid = (l + r) / 2;
            self(self, 2 * p, l, mid);
            self(self, 2 * p + 1, mid, r);
            pull(p);
        };

        build(build, 1, 0, n);
    }

    void pull(int p) {
        info[p] = Mono::op(info[2 * p], info[2 * p + 1]);
    }

    void apply(int p, int len, const Y& v) {
        info[p] = ActMono::act(info[p], v, len);
        tag[p] = Tag::op(tag[p], v);
    }

    void push(int p, int len) {
```

```
        apply(2 * p, len / 2, tag[p]);
        apply(2 * p + 1, (len + 1) / 2, tag[p]);
        tag[p] = Tag::e();
    }

    void modify(int p, int l, int r, int x, const X& v) {
        if (r - l == 1) {
            info[p] = v;
            return;
        }
        int mid = (l + r) / 2;
        push(p, r - l);
        if (x < mid) {
            modify(2 * p, l, mid, x, v);
        } else {
            modify(2 * p + 1, mid, r, x, v);
        }
        pull(p);
    }
    void modify(int p, const X& v) {
        modify(1, 0, n, p, v);
    }

    X rangeQuery(int p, int l, int r, int x, int y) {
        if (y <= l || r <= x) {
            return Mono::e();
        }
        if (x <= l && r <= y) {
            return info[p];
        }
        int mid = (l + r) / 2;
        push(p, r - l);
        return Mono::op(rangeQuery(2 * p, l, mid, x, y), rangeQuery(2 * p + 1, mid,
r, x, y));
    }
    X rangeQuery(int l, int r) {
        return rangeQuery(1, 0, n, l, r);
    }

    void rangeApply(int p, int l, int r, int x, int y, const Y& v) {
        if (y <= l || r <= x) {
            return;
        }
        if (x <= l && r <= y) {
            apply(p, r - l, v);
            return;
        }
        int mid = (l + r) / 2;
        push(p, r - l);
        rangeApply(2 * p, l, mid, x, y, v);
        rangeApply(2 * p + 1, mid, r, x, y, v);
        pull(p);
    }
    void rangeApply(int l, int r, const Y& v) {
        return rangeApply(1, 0, n, l, r, v);
    }

    template <typename F>
    int findFirst(int p, int l, int r, int x, int y, F&& pred) {
        if (y <= l || r <= x) {
            return -1;
        }
        if (x <= l && r <= y && !pred(info[p])) {
            return -1;
        }
        if (r - l == 1) {
            return l;
        }
        int mid = (l + r) / 2;
        push(p, r - l);
        int res = findFirst(2 * p, l, mid, x, y, pred);
        if (res == -1) {
            res = findFirst(2 * p + 1, mid, r, x, y, pred);
        }
        return res;
    }
    template <typename F>
    int findFirst(int l, int r, F&& pred) {
        return findFirst(1, 0, n, l, r, pred);
    }

    template <typename F>
    int findLast(int p, int l, int r, int x, int y, F&& pred) {
        if (y <= l || r <= x) {
            return -1;
        }
```

```
        if (x <= l && r <= y && !pred(info[p])) {
            return -1;
        }
        if (r - l == 1) {
            return l;
        }
        int mid = (l + r) / 2;
        push(p, r - l);
        int res = findLast(2 * p + 1, mid, r, x, y, pred);
        if (res == -1) {
            res = findLast(2 * p, l, mid, x, y, pred);
        }
        return res;
    }
    template <typename F>
    int findLast(int l, int r, F&& pred) {
        return findLast(1, 0, n, l, r, pred);
    }

    template <typename F>
    int maxRight(int l, F&& pred) {
        X sum = Mono::e();
        int p = findFirst(l, n, [&](const X& x) {
            X v = Mono::op(sum, x);
            if (!pred(v)) {
                return true;
            }
            sum = v;
            return false;
        });
        return p == -1 ? n : p;
    }

    template <typename F>
    int minLeft(int r, F&& pred) {
        X sum = Mono::e();
        int p = findLast(0, r, [&](const X& x) {
            X v = Mono::op(x, sum);
            if (!pred(v)) {
                return true;
            }
            sum = v;
            return false;
        });
        return p + 1;
    }
};
```

7.1.3 动态开点线段树

```
template <typename ActMono>
struct DynamicSegmentTree {
    using Mono = typename ActMono::Mono;
    using Tag = typename ActMono::Tag;
    using X = typename ActMono::X;
    using Y = typename ActMono::Y;

    int n;
    int root;
    vector<X> info;
    vector<Y> tag;
    vector<int> lc, rc;

    DynamicSegmentTree() : n(0), root(-1) {}
    DynamicSegmentTree(int n_) {
        init(n_);
    }

    void init(int n_) {
        n = n_;
        info.clear();
        tag.clear();
        lc.clear();
        rc.clear();
        root = newNode();
    }

    int newNode() {
        info.push_back(Mono::e());
        tag.push_back(Tag::e());
        lc.push_back(-1);
        rc.push_back(-1);
        return int(info.size()) - 1;
    }

    void pull(int p) {
        info[p] = Mono::op(info[lc[p]], info[rc[p]]);
```

```
    }

    void apply(int p, int len, const Y& v) {
        info[p] = ActMono::act(info[p], v, len);
        tag[p] = Tag::op(tag[p], v);
    }

    void push(int p, int len) {
        if (lc[p] == -1) {
            lc[p] = newNode();
        }
        if (rc[p] == -1) {
            rc[p] = newNode();
        }
        apply(lc[p], len / 2, tag[p]);
        apply(rc[p], (len + 1) / 2, tag[p]);
        tag[p] = Tag::e();
    }

    void modify(int p, int l, int r, int x, const X& v) {
        if (r - l == 1) {
            info[p] = v;
            return;
        }
        int mid = (l + r) / 2;
        push(p, r - l);
        if (x < mid) {
            modify(lc[p], l, mid, x, v);
        } else {
            modify(rc[p], mid, r, x, v);
        }
        pull(p);
    }
    void modify(int x, const X& v) {
        modify(root, 0, n, x, v);
    }

    X rangeQuery(int p, int l, int r, int x, int y) {
        if (y <= l || r <= x) {
            return Mono::e();
        }
        if (x <= l && r <= y) {
            return info[p];
        }
        int mid = (l + r) / 2;
        push(p, r - l);
        return Mono::op(rangeQuery(lc[p], l, mid, x, y), rangeQuery(rc[p], mid, r,
x, y));
    }
    X rangeQuery(int l, int r) {
        return rangeQuery(root, 0, n, l, r);
    }

    void rangeApply(int p, int l, int r, int x, int y, const Y& v) {
        if (y <= l || r <= x) {
            return;
        }
        if (x <= l && r <= y) {
            apply(p, r - l, v);
            return;
        }
        int mid = (l + r) / 2;
        push(p, r - l);
        rangeApply(lc[p], l, mid, x, y, v);
        rangeApply(rc[p], mid, r, x, y, v);
        pull(p);
    }
    void rangeApply(int l, int r, const Y& v) {
        rangeApply(root, 0, n, l, r, v);
    }

    template <typename F>
    int findFirst(int p, int l, int r, int x, int y, F&& pred) {
        if (y <= l || r <= x) {
            return -1;
        }
        if (x <= l && r <= y && !pred(info[p])) {
            return -1;
        }
        if (r - l == 1) {
            return l;
        }
        int mid = (l + r) / 2;
        push(p, r - l);
        int res = findFirst(lc[p], l, mid, x, y, pred);
        if (res == -1) {
```

```
            res = findFirst(rc[p], mid, r, x, y, pred);
        }
        return res;
    }
    template <typename F>
    int findFirst(int l, int r, F&& pred) {
        return findFirst(root, 0, n, l, r, pred);
    }

    template <typename F>
    int findLast(int p, int l, int r, int x, int y, F&& pred) {
        if (y <= l || r <= x) {
            return -1;
        }
        if (x <= l && r <= y && !pred(info[p])) {
            return -1;
        }
        if (r - l == 1) {
            return l;
        }
        int mid = (l + r) / 2;
        push(p, r - l);
        int res = findLast(rc[p], mid, r, x, y, pred);
        if (res == -1) {
            res = findLast(lc[p], l, mid, x, y, pred);
        }
        return res;
    }
    template <typename F>
    int findLast(int l, int r, F&& pred) {
        return findLast(root, 0, n, l, r, pred);
    }

    template <typename F>
    int maxRight(int l, F&& pred, X& sum) {
        int p = findFirst(l, n, [&](const X& x) {
            X v = Mono::op(sum, x);
            if (!pred(v)) {
                return true;
            }
            sum = v;
            return false;
        });
        return p == -1 ? n : p;
    }

    template <typename F>
    int minLeft(int r, F&& pred, X& sum) {
        int p = findLast(0, r, [&](const X& x) {
            X v = Mono::op(x, sum);
            if (!pred(v)) {
                return true;
            }
            sum = v;
            return false;
        });
        return p + 1;
    }
};
```

7.2 树状数组

7.2.1 树状数组

```
template <typename Mono>
struct Fenwick {
    using X = typename Mono::Type;

    int n;
    vector<X> a;

    Fenwick() : n(0) {}
    Fenwick(int n_) {
        init(n_);
    }

    void init(int n_) {
        n = n_;
        a.assign(n, Mono::e());
    }

    void add(int x, const X& v) {
        assert(0 <= x && x < n);
        for (int i = x + 1; i <= n; i += i & -i) {
            a[i - 1] = Mono::op(a[i - 1], v);
        }
    }

```

```
    X sum(int x) {
        assert(0 <= x && x <= n);
        X res = Mono::e();
        for (int i = x; i > 0; i -= i & -i) {
            res = Mono::op(a[i - 1], res);
        }
        return res;
    }

    X rangeSum(int l, int r) {
        assert(l <= r);
        return Mono::op(Mono::inv(sum(l)), sum(r));
    }

    // lower_bound
    int kth(X k) {
        int x = 0;
        for (int i = 1 << __lg(n); i; i /= 2) {
            if (x + i <= n && k >= a[x + i - 1]) {
                x += i;
                k -= a[x - 1];
            }
        }
        return x;
    }
};
```

7.2.2 区间加、单点查树状数组

```
template <typename Abel>
struct DualFenwick {
    using X = typename Abel::Type;

    int n;
    Fenwick<Abel> fen;

    DualFenwick() : n(0), fen(0) {}
    DualFenwick(int n_) {
        init(n_);
    }

    void init(int n_) {
        n = n_;
        fen.init(n);
    }

    void add(int l, int r, const X& v) {
        assert(0 <= l && l <= r && r <= n);
        fen.add(l, v);
        if (r < n) {
            fen.add(r, Abel::inv(v));
        }
    }

    X query(int x) {
        assert(0 <= x && x < n);
        return fen.sum(x + 1);
    }
};
```

7.2.3 区间加、区间和树状数组

```
template <typename Abel>
struct FenwickRangeAdd {
    using X = typename Abel::Type;

    int n;
    Fenwick<Abel> fen0, fen1;

    FenwickRangeAdd() : n(0), fen0(1), fen1(1) {}
    FenwickRangeAdd(int n_) {
        init(n_);
    }

    void init(int n_) {
        n = n_;
        fen0.init(n + 1);
        fen1.init(n + 1);
    }

    void add(int l, int r, const X& v) {
        assert(0 <= l && l <= r && r <= n);
        fen0.add(l, Abel::power(v, -static_cast<i64>(l)));
        fen0.add(r, Abel::power(v, static_cast<i64>(r)));
        fen1.add(l, v);
        fen1.add(r, Abel::inv(v));
    }

    X sum(int r) {
```

```
        assert(0 <= r && r <= n);
        return Abel::op(Abel::power(fen1.sum(r), static_cast<i64>(r)),
fen0.sum(r));
    }

    X rangeSum(int l, int r) {
        assert(0 <= l && l <= r && r <= n);
        return Abel::op(Abel::inv(sum(l)), sum(r));
    }
};
```

7.2.4 二维树状数组

```
template <typename Mono>
struct Fenwick2D {
    using X = typename Mono::Type;

    int h, w;
    vector<X> a;

    Fenwick2D() : h(0), w(0) {}
    Fenwick2D(int h_, int w_) {
        init(h_, w_);
    }

    void init(int h_, int w_) {
        h = h_;
        w = w_;
        a.assign((h + 1) * (w + 1), Mono::e());
    }

    void add(int x, int y, const X& v) {
        assert(0 <= x && x < h);
        assert(0 <= y && y < w);
        for (int i = x + 1; i <= h; i += i & -i) {
            for (int j = y + 1; j <= w; j += j & -j) {
                a[i * (w + 1) + j] = Mono::op(a[i * (w + 1) + j], v);
            }
        }
    }

    X sum(int x, int y) {
        assert(0 <= x && x <= h);
        assert(0 <= y && y <= w);
        X res = Mono::e();
        for (int i = x; i > 0; i -= i & -i) {
            for (int j = y; j > 0; j -= j & -j) {
                res = Mono::op(res, a[i * (w + 1) + j]);
            }
        }
        return res;
    }

    X rangeSum(int x1, int x2, int y1, int y2) {
        assert(0 <= x1 && x1 <= x2 && x2 <= h);
        assert(0 <= y1 && y1 <= y2 && y2 <= w);
        X s22 = sum(x2, y2);
        X s12 = sum(x1, y2);
        X s21 = sum(x2, y1);
        X s11 = sum(x1, y1);
        return Mono::op(Mono::op(s22, Mono::inv(s12)), Mono::op(Mono::inv(s21),
s11));
    }
};
```

7.3 倍增

```
template <typename Mono>
struct BinaryLifting {
    using X = typename Mono::Type;

    int n, logn;
    vector<vector<int>> nxt;
    vector<vector<X>> info;

    BinaryLifting() : n(0) {}
    BinaryLifting(int n_) {
        init(n_);
    }

    void init(int n_) {
        n = n_;
        logn = __lg(n);
        nxt.assign(logn + 1, vector<int>(n, -1));
        info.assign(logn + 1, vector<X>(n, Mono::e()));
    }

    void add(int u, int v, const X& x) {
        assert(-1 <= v && v < n);
```

```
        nxt[0][u] = v;
        info[0][u] = x;
    }

    void build() {
        for (int k = 0; k < logn; k++) {
            for (int u = 0; u < n; u++) {
                int v = nxt[k][u];
                if (v == -1) {
                    nxt[k + 1][u] = -1;
                    info[k + 1][u] = info[k][u];
                } else {
                    nxt[k + 1][u] = nxt[k][v];
                    info[k + 1][u] = Mono::op(info[k][u], info[k][v]);
                }
            }
        }
    }

    pair<int, X> query(int u, i64 step) {
        assert(0 <= step && step < (1LL << (logn + 1)));
        X res = Mono::e();
        for (int k = logn; k >= 0 && u != -1; k--) {
            if ((step >> k) & 1) {
                res = Mono::op(res, info[k][u]);
                u = nxt[k][u];
            }
        }
        return {u, res};
    }

    int jump(int u, i64 step) {
        assert(0 <= step && step < (1LL << (logn + 1)));
        for (int k = logn; k >= 0 && u != -1; k--) {
            if ((step >> k) & 1) {
                u = nxt[k][u];
            }
        }
        return u;
    }
};
```

7.4 并查集

```
struct DSU {
    vector<int> f, siz;

    DSU() {}
    DSU(int n) {
        init(n);
    }

    void init(int n) {
        f.resize(n);
        iota(f.begin(), f.end(), 0);
        siz.assign(n, 1);
    }

    int find(int x) {
        while (x != f[x]) {
            x = f[x] = f[f[x]];
        }
        return x;
    }

    bool same(int x, int y) {
        return find(x) == find(y);
    }

    bool merge(int x, int y) {
        x = find(x);
        y = find(y);
        if (x == y) {
            return false;
        }
        siz[x] += siz[y];
        f[y] = x;
        return true;
    }

    int size(int x) {
        return siz[find(x)];
    }
};
```

7.5 稀疏表

```cpp
template <typename Mono>
struct SparseTable {
    using X = typename Mono::Type;

    int n;
    vector<vector<X>> st;

    SparseTable() : n(0) {}
    SparseTable(const vector<X>& a) {
        init(a);
    }

    void init(const vector<X>& a) {
        n = a.size();
        int logn = __lg(n);
        st.assign(logn + 1, vector<X>(n));
        for (int i = 0; i < n; i++) {
            st[0][i] = a[i];
        }
        for (int j = 1; (1 << j) <= n; j++) {
            for (int i = 0; i + (1 << j) <= n; i++) {
                st[j][i] = Mono::op(st[j - 1][i], st[j - 1][i + (1 << (j - 1))]);
            }
        }
    }

    X query(int l, int r) {
        assert(l < r);
        int len = r - l;
        int t = __lg(len);
        return Mono::op(st[t][l], st[t][r - (1 << t)]);
    }
};
```

7.6 01-Trie

```cpp
struct BinaryTrie {
    struct Node {
        array<int, 2> ch;
        int end;
        Node() {
            ch[0] = ch[1] = 0;
            end = 0;
        }
    };

    vector<Node> tr;
    int tot;

    BinaryTrie(int n) {
        init(n);
    }

    void init(int n) {
        tr.resize(n);
        tot = 1;
    }

    void insert(int x) {
        int p = 1;
        for (int i = 30; i >= 0; i--) {
            int c = x >> i & 1;
            if (!tr[p].ch[c]) {
                ++tot;
                tr[p].ch[c] = tot;
            }
            p = tr[p].ch[c];
            tr[p].end++;
        }
    }

    int query(int x) {
        int p = 1, res = 0;
        for (int i = 30; i >= 0; i--) {
            int c = x >> i & 1;
            if (tr[p].ch[c ^ 1]) {
                res |= (1 << i);
                p = tr[p].ch[c ^ 1];
            } else {
                p = tr[p].ch[c];
            }
        }
        return res;
    }

    void erase(int x) {
        int p = 1;
```

```
        for (int i = 30; i >= 0; i--) {
            int c = x >> i & 1;
            int nxt = tr[p].ch[c];
            tr[nxt].end--;
            if (tr[nxt].end == 0) {
                tr[p].ch[c] = 0;
            }
            p = nxt;
        }
    }
};

```

7.7 笛卡尔树

```
template <typename T, bool IS_MIN = false>
struct CartesianTree {
    int n;
    vector<T>& a;
    vector<int> L, R, p;
    // range[i]: the interval of the subtree rooted at i
    vector<pair<int, int>> range;
    int root;

    CartesianTree(vector<T>& a_) : n(a_.size()), a(a_), root(-1) {
        L.assign(n, -1);
        R.assign(n, -1);
        p.assign(n, -1);
        range.assign(n, {0, n});
        vector<int> stk;

        auto cmp = [&](int i, int j) -> bool {
            if (a[i] == a[j]) {
                return i < j;
            }
            if constexpr (IS_MIN) {
                return a[i] < a[j];
            } else {
                return a[j] < a[i];
            }
        };

        for (int i = 0; i < n; i++) {
            while (!stk.empty() && cmp(i, stk.back())) {
                L[i] = stk.back();
                stk.pop_back();
            }
            range[i].first = (stk.empty() ? 0 : stk.back() + 1);
            if (!stk.empty()) {
                R[stk.back()] = i;
            }
            stk.push_back(i);
        }
        if (!stk.empty()) {
            root = stk.front();
        }
        stk.clear();
        for (int i = n - 1; i >= 0; i--) {
            while (!stk.empty() && cmp(i, stk.back())) {
                stk.pop_back();
            }
            range[i].second = stk.empty() ? n : stk.back();
            stk.push_back(i);
        }
        for (int i = 0; i < n; i++) {
            if (L[i] != -1) {
                p[L[i]] = i;
            }
            if (R[i] != -1) {
                p[R[i]] = i;
            }
        }
    }
};
```

8 线性代数

8.1 矩阵

```
template <typename T>
struct Matrix {
    int n, m;
    vector<vector<T>> a;

    Matrix() : n(0), m(0) {}
    Matrix(int n_, int m_, T v = T{}) : n(n_), m(m_), a(n, vector<T>(m, v)) {}
    Matrix(int n_) : Matrix(n_, n_) {}

```

```
    static Matrix identity(int n) {
        Matrix res(n);
        for (int i = 0; i < n; i++) {
            res[i][i] = T{1};
        }
        return res;
    }

    vector<T>& operator[](int i) {
        return a[i];
    }
    const vector<T>& operator[](int i) const {
        return a[i];
    }
};

template <typename T>
Matrix<T> operator*(const Matrix<T>& a, const Matrix<T>& b) {
    assert(a.m == b.n);
    Matrix<T> c(a.n, b.m);
    for (int i = 0; i < c.n; i++) {
        for (int k = 0; k < a.m; k++) {
            for (int j = 0; j < c.m; j++) {
                c.a[i][j] += a.a[i][k] * b.a[k][j];
            }
        }
    }
    return c;
}

template <typename T>
Matrix<T>& operator*=(Matrix<T>& a, const Matrix<T>& b) {
    return a = a * b;
}

template <typename T, integral U>
Matrix<T> power(Matrix<T> a, U b) {
    Matrix<T> res = Matrix<T>::identity(a.n);
    for (; b > 0; b >>= 1) {
        if (b & 1) {
            res *= a;
        }
        a *= a;
    }
    }
    return res;
}

template <typename T>
Matrix<T> transpose(const Matrix<T>& a) {
    Matrix<T> res(a.m, a.n);
    for (int i = 0; i < a.n; i++) {
        for (int j = 0; j < a.m; j++) {
            res[j][i] = a[i][j];
        }
    }
    return res;
}

template <typename T>
Matrix<T> augment(const Matrix<T>& a, const Matrix<T>& b) {
    assert(a.n == b.n);
    Matrix<T> res(a.n, a.m + b.m);
    for (int i = 0; i < a.n; i++) {
        for (int j = 0; j < a.m; j++) {
            res[i][j] = a[i][j];
        }
        for (int j = 0; j < b.m; j++) {
            res[i][a.m + j] = b[i][j];
        }
    }
    return res;
}

template <typename T>
Matrix<T> augment(const Matrix<T>& a, const vector<T>& b) {
    assert(a.n == int(b.size()));
    Matrix<T> res(a.n, a.m + 1);
    for (int i = 0; i < a.n; i++) {
        for (int j = 0; j < a.m; j++) {
            res[i][j] = a[i][j];
        }
        res[i][a.m] = b[i];
    }
    return res;
}
```

8.2 高斯消元

```cpp
template <typename T>
pair<vector<int>, T> eliminate(Matrix<T>& A, int lim, bool full) {
    // full == true -> RREF
    // full == false -> REF
    vector<int> pivot;
    T prod = T{1};
    int r = 0;
    for (int j = 0; j < lim && r < A.n; j++) {
        int p = r;
        while (p < A.n && A[p][j] == T{0}) {
            p++;
        }
        if (p == A.n) {
            continue;
        }
        if (p != r) {
            swap(A[p], A[r]);
            prod = -prod;
        }
        T x = A[r][j];
        prod *= x;
        pivot.push_back(j);
        T inv = T{1} / x;
        for (int k = j; k < A.m; k++) {
            A[r][k] *= inv;
        }
        for (int i = 0; i < A.n; i++) {
            if (i == r || (!full && i < r)) {
                continue;
            }
            T c = A[i][j];
            if (c == T{0}) {
                continue;
            }
            A[i][j] = T{0};
            for (int k = j + 1; k < A.m; k++) {
                A[i][k] -= c * A[r][k];
            }
        }
        r++;
    }
    return {pivot, prod};
}

template <typename T>
pair<vector<int>, T> gauss(Matrix<T>& A) {
    return eliminate(A, A.m, false);
}
template <typename T>
pair<vector<int>, T> gaussJordan(Matrix<T>& A, int lim) {
    return eliminate(A, lim, true);
}

template <typename T>
pair<T, Matrix<T>> matrixInv(const Matrix<T>& a) {
    if (a.n != a.m) {
        return {T{0}, {}};
    }
    int n = a.n;
    auto aug = augment(a, Matrix<T>::identity(n));
    auto [pivot, det] = gaussJordan(aug, n);
    if (pivot.size() < n) {
        return {T{0}, {}};
    }
    Matrix<T> inv(n);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            inv[i][j] = aug[i][n + j];
        }
    }
    return {det, inv};
}

template <typename T>
int matrixRank(Matrix<T> a) {
    auto [pivot, prod] = gauss(a);
    return pivot.size();
}

template <typename T>
T determinant(Matrix<T> A) {
    assert(A.n == A.m);
    auto [pivot, det] = gauss(A);
    if (pivot.size() < A.n) {
```

```
        return T{0};
    }
    return det;
}

template <typename T>
vector<vector<T>> solveLinear(const Matrix<T>& A, const vector<T>& b) {
    int n = A.n;
    int m = A.m;
    auto aug = augment(A, b);
    auto [pivot, prod] = gaussJordan(aug, m);
    int r = pivot.size();
    for (int i = r; i < n; i++) {
        if (aug[i][m] != T{0}) {
            return {};
        }
    }
    vector<int> row(m, -1);
    for (int i = 0; i < r; i++) {
        row[pivot[i]] = i;
    }
    vector<vector<T>> sol(1, vector<T>(m));
    for (int j = 0; j < m; j++) {
        if (row[j] != -1) {
            sol[0][j] = aug[row[j]][m];
        }
    }
    for (int j = 0; j < m; j++) {
        if (row[j] != -1) {
            continue;
        }
        vector<T> x(m);
        x[j] = T{-1};
        for (int i = 0; i < r; i++) {
            x[pivot[i]] = aug[i][j];
        }
        sol.push_back(x);
    }
    return sol;
}
```

8.3 GF(2) 空间

8.3.1 异或线性基

```
struct Basis {
    array<u64, 64> p{};
    bool norm = false;

    int size() {
        int r = 0;
        for (int i = 0; i < 64; i++) {
            r += p[i] != 0;
        }
        return r;
    }

    bool insert(u64 x) {
        for (int i = 63; i >= 0; i--) {
            if (!(x >> i & 1)) {
                continue;
            }
            if (!p[i]) {
                p[i] = x;
                norm = false;
                return true;
            }
            x ^= p[i];
        }
        return false;
    }

    bool contain(u64 x) {
        for (int i = 63; i >= 0; i--) {
            if (x >> i & 1) {
                x ^= p[i];
            }
        }
        return x == 0;
    }

    u64 getMax(u64 x = 0) {
        for (int i = 63; i >= 0; i--) {
            if ((x ^ p[i]) > x) {
                x ^= p[i];
            }
```

```
        }
        return x;
    }

    u64 getMin(u64 x = 0) {
        for (int i = 63; i >= 0; i--) {
            if ((x ^ p[i]) < x) {
                x ^= p[i];
            }
        }
        return x;
    }

    void normalize() {
        if (norm == true) {
            return;
        }
        norm = true;
        for (int i = 63; i >= 0; i--) {
            if (!p[i]) {
                continue;
            }
            for (int j = i - 1; j >= 0; j--) {
                if (p[i] >> j & 1) {
                    p[i] ^= p[j];
                }
            }
        }
    }

    u64 kth(u64 k) {
        normalize();
        vector<u64> b;
        for (int i = 0; i < 64; i++) {
            if (p[i]) {
                b.push_back(p[i]);
            }
        }
        int r = b.size();
        if (r < 64) {
            if (k >= (1ULL << r)) {
                return -1;
            }
        }
        u64 res = 0;
        for (int i = 0; i < r; i++) {
            if (k >> i & 1) {
                res ^= b[i];
            }
        }
        return res;
    }
};
```

8.3.2 逆矩阵

```
namespace gf2 {

template <typename U>
vector<U> inverse(int n, vector<U> a) {
    vector<U> b(n);
    for (int i = 0; i < n; i++) {
        b[i] = U{1} << i;
    }
    for (int j = 0; j < n; j++) {
        int p = j;
        while (p < n && !(a[p] >> j & U{1})) {
            p++;
        }
        if (p == n) {
            return {};
        }
        swap(a[p], a[j]);
        swap(b[p], b[j]);
        for (int i = 0; i < n; i++) {
            if (i != j && (a[i] >> j & U{1})) {
                a[i] ^= a[j];
                b[i] ^= b[j];
            }
        }
    }
    return b;
}

}
```

8.3.3 转置

```cpp
namespace gf2 {

template <typename U>
vector<U> transpose(int n, int m, const vector<U>& a) {
    vector<U> b(m);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (a[i] >> j & U{1}) {
                b[j] |= U{1} << i;
            }
        }
    }
    return b;
}

}
```

8.3.4 GF(2) 线性方程组

```cpp
namespace gf2 {

template <unsigned_integral U>
vector<U> solveLinear(int n, int m, vector<U> a, U b) {
    vector<U> mat(m);
    U rhs = 0;
    for (int i = 0; i < n; i++) {
        U x = a[i], c = b >> i & 1;
        for (int j = m - 1; j >= 0; j--) {
            if ((x ^ mat[j]) < x) {
                x ^= mat[j];
                c ^= rhs >> j & 1;
            }
        }
        if (!x) {
            if (c) {
                return {};
            }
            continue;
        }
        int k = bit_width(x) - 1;
        for (int j = k; j < m; j++) {
            if ((mat[j] ^ x) < mat[j]) {
                mat[j] ^= x;
                rhs ^= c << j;
            }
        }
        mat[k] = x;
        rhs |= c << k;
    }
    vector<U> res{rhs};
    mat = transpose(m, m, mat);
    for (int j = 0; j < m; j++) {
        if (!(mat[j] >> j & 1)) {
            res.push_back(mat[j] | (U{1} << j));
        }
    }
    return res;
}

}
```

9 字符串

9.1 Manacher

```cpp
vector<int> manacher(string s) {
    string t = "#";
    for (auto c : s) {
        t += c;
        t += '#';
    }
    int n = t.size();
    vector<int> r(n);
    for (int i = 0, j = 0; i < n; i++) {
        if (2 * j - i >= 0 && j + r[j] > i) {
            r[i] = min(r[2 * j - i], j + r[j] - i);
        }
        while (i - r[i] >= 0 && i + r[i] < n && t[i - r[i]] == t[i + r[i]]) {
            r[i] += 1;
        }
        if (i + r[i] > j + r[j]) {
            j = i;
        }
    }
    return r;
}
```

9.2 KMP

```cpp
vector<int> kmp(string s) {
    int n = s.size();
    vector<int> f(n + 1);
    for (int i = 1, j = 0; i < n; i++) {
        while (j && s[i] != s[j]) {
            j = f[j];
        }
        j += (s[i] == s[j]);
        f[i + 1] = j;
    }
    return f;
}

```

9.3 Trie

```cpp
struct Trie {
    struct Node {
        array<int, 26> nxt;
        int pass, end;
        Node() {
            nxt.fill(-1);
            end = 0;
            pass = 0;
        }
    };

    vector<Node> tr;
    int tot;

    Trie(int n) {
        init(n);
    }

    void init(int n) {
        tr.assign(n, {});
        tot = 0;
    }

    void insert(string& s) {
        int p = 0;
        ++tr[p].pass;
        for (char& ch : s) {
            int c = ch - 'a';
            if (tr[p].nxt[c] == -1) {
                ++tot;
                tr[p].nxt[c] = tot;
            }
            p = tr[p].nxt[c];
            ++tr[p].pass;
        }
        ++tr[p].end;
    }

    int count(string& s) {
        int p = 0;
        for (char& ch : s) {
            int c = ch - 'a';
            if (tr[p].nxt[c] == -1) {
                return false;
            }
            p = tr[p].nxt[c];
        }
        return tr[p].end;
    }
};

```

9.4 Z 函数

```cpp
vector<int> Z(string s) {
    int n = s.size();
    vector<int> z(n + 1);
    z[0] = n;
    for (int i = 1, j = 1; i < n; i++) {
        z[i] = max(0, min(j + z[j] - i, z[i - j]));
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if (i + z[i] > j + z[j]) {
            j = i;
        }
    }
    return z;
}

```

9.5 AC 自动机

```
struct AhoCorasick {
    static constexpr int ALPHABET = 26;
    struct Node {
        int len;
        int link;
        array<int, ALPHABET> next;
        Node() : len{0}, link{0}, next{} {}
    };

    vector<Node> t;

    AhoCorasick() {
        init();
    }

    void init() {
        t.assign(2, Node());
        t[0].next.fill(1);
        t[0].len = -1;
    }

    int newNode() {
        t.emplace_back();
        return t.size() - 1;
    }

    int add(const string &a) {
        int p = 1;
        for (auto c : a) {
            int x = c - 'a';
            if (t[p].next[x] == 0) {
                t[p].next[x] = newNode();
                t[t[p].next[x]].len = t[p].len + 1;
            }
            p = t[p].next[x];
        }
        return p;
    }

    void work() {
        queue<int> q;
        q.push(1);

        while (!q.empty()) {
            int x = q.front();
            q.pop();

            for (int i = 0; i < ALPHABET; i++) {
                if (t[x].next[i] == 0) {
                    t[x].next[i] = t[t[x].link].next[i];
                } else {
                    t[t[x].next[i]].link = t[t[x].link].next[i];
                    q.push(t[x].next[i]);
                }
            }
        }
    }

    int next(int p, int x) {
        return t[p].next[x];
    }

    int link(int p) {
        return t[p].link;
    }

    int len(int p) {
        return t[p].len;
    }

    int size() {
        return t.size();
    }
};
```

10 杂项

10.1 debug

```
using namespace std;

#define dbg(...) { cerr << "[" << __LINE__ << "] "; pr(__VA_ARGS__); cerr << '\n'; }

void pr(bool b) { cerr << (b ? "T" : "F"); }
void pr(char c) { cerr << "'" << c << "'"; }
void pr(string s) { cerr << '"' << s << '"'; }
```

```
void pr(const char* s) { cerr << '"' << s << '"'; }

template <class T>
void pr(T&& x) {
    using U = remove_reference_t<T>;
    if constexpr (requires { begin(x); }) {
        bool br = requires { typename U::key_type; };
        cerr << (br ? '{' : '[');
        bool f = 0;
        for (auto&& e : x) {
            if (exchange(f, 1)) cerr << ",";
            if constexpr (requires { typename U::mapped_type; }) { pr(e.first); cerr << ":"; pr(e.second); }
            else pr(e);
        }
        cerr << (br ? '}' : ']');
    } else if constexpr (requires { x.first; x.second; }) {
        cerr << '('; pr(x.first); cerr << ","; pr(x.second); cerr << ')';
    } else if constexpr (requires { get<0>(x); }) {
        cerr << '(';
        bool f = 0;
        apply([&](auto... e) { ((cerr << (exchange(f, 1) ? "," : ""), pr(e)), ...); }, x);
        cerr << ')';
    } else if constexpr (requires { x.top(); }) {
        vector<typename U::value_type> v;
        while (!x.empty()) v.push_back(x.top()), x.pop();
        pr(v);
    } else if constexpr (requires { x.front(); }) {
        vector<typename U::value_type> v;
        while (!x.empty()) v.push_back(x.front()), x.pop();
        pr(v);
    } else {
        cerr << x;
    }
}

template <class A, class... B> requires (sizeof...(B) > 0)
void pr(A a, B... b) { pr(a); ((cerr << "  ", pr(b)), ...); }
```

10.2 template

```
using namespace std;

using i64 = int64_t;
using u64 = uint64_t;
using u32 = uint32_t;

using i128 = __int128_t;
using u128 = __uint128_t;

template <typename T>
constexpr T inf = numeric_limits<T>::max() / 2;
template <>
constexpr int inf<int> = 1e9;
template <>
constexpr i64 inf<i64> = 1e18;

void solve();

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int t;
    cin >> t;

    while (t--) {
        solve();
    }

    return 0;
}
```

10.3 二分答案

```
template <typename T, typename F>
T firstTrue(T lo, T hi, F&& pred) {
    while (lo < hi) {
        T mid = (lo + hi) / 2;
        if (pred(mid)) {
            hi = mid;
        } else {
            lo = mid + 1;
        }
    }
    return lo;
}

```

```cpp
template <typename T, typename F>
T lastTrue(T lo, T hi, F&& pred) {
    while (lo < hi) {
        T mid = (lo + hi + 1) / 2;
        if (pred(mid)) {
            lo = mid;
        } else {
            hi = mid - 1;
        }
    }
    return lo;
}
```

10.4 i128

```cpp
using i128 = __int128;

ostream& operator<<(ostream& os, i128 n) {
    if (n == 0) {
        return os << 0;
    }
    bool neg = n < 0;
    u128 x = neg ? u128(0) - u128(n) : u128(n);
    string s;
    while (x > 0) {
        s += char('0' + x % 10);
        x /= 10;
    }
    if (neg) s += '-';
    reverse(s.begin(), s.end());
    return os << s;
}

i128 toi128(const string& s) {
    bool neg = !s.empty() && s[0] == '-';
    int start = !s.empty() && (s[0] == '-' || s[0] == '+');
    i128 n = 0;
    for (int i = start; i < int(s.size()); i++) {
        int d = s[i] - '0';
        n = neg ? n * 10 - d : n * 10 + d;
    }
    return n;
}

i128 sqrti128(i128 n) {
    assert(n >= 0);
    i128 lo = 0, hi = i128(1) << 64;
    while (lo < hi) {
        i128 x = (lo + hi + 1) / 2;
        if (x <= n / x) {
            lo = x;
        } else {
            hi = x - 1;
        }
    }
    return lo;
}

i128 gcd(i128 a, i128 b) {
    while (b) {
        a %= b;
        swap(a, b);
    }
    return a;
}
```

10.5 日期

```cpp
const int d[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};

bool isLeap(int y) {
    return y % 400 == 0 || (y % 4 == 0 && y % 100 != 0);
}

int daysInMonth(int y, int m) {
    return d[m - 1] + (isLeap(y) && m == 2);
}

int getDay(int y, int m, int d) {
    int ans = 0;
    for (int i = 1970; i < y; i++) {
        ans += 365 + isLeap(i);
    }
    for (int i = 1; i < m; i++) {
        ans += daysInMonth(y, i);
    }
    ans += d;
    return (ans + 2) % 7 + 1;
```

```
}

```